



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Titolo della Tesi

RELATORE

Prof. Fabio Palomba

Dott. Nome tutor

Università degli Studi di Salerno

CANDIDATO

Marco Donatiello

Matricola: 0512119045

Anno Accademico 2025-2026

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

Dedica o citazione

Abstract

Il testo dell'abstract qui...

Indice

Elenco delle Figure	iii
Elenco delle Tabelle	iv
1 Introduzione	1
2 Analisi dello stato del arte e contesto tecnologico	2
2.1 OWASP Cloud Security Top 10	2
2.1.1 Security Misconfiguration	4
2.2 API Security Risks	6
2.2.1 API1:2023 – Broken Object Level Authorization	8
2.2.2 API2:2023 – Broken Authentication	14
2.2.3 API3:2023 – Broken Object Property Level Authorization	16
2.2.4 API4:2023 – Unrestricted Resource Consumption	19
2.2.5 API5:2023 – Broken Function Level Authorization	28
2.2.6 API6:2023 – Unrestricted Access to Sensitive Business Flows	33
2.2.7 API7:2023 – Server Side Request Forgery	35
2.2.8 API8:2023 – Security Misconfiguration	38
2.2.9 API10:2023 – Unsafe Consumption of APIs	47
3 Conclusioni	55

Bibliografia

56

Elenco delle figure

Elenco delle tabelle

2.1	Relazioni fra API4:2023 e le categorie contigue della OWASP API Security Top 10 2023.	21
2.2	Relazioni fra API5:2023 e le altre categorie della OWASP API Security Top 10 2023.	29
2.3	Relazioni fra API7:2023 e le categorie contigue della OWASP API Security Top 10 2023.	35
2.4	Sovrapposizioni fra API8:2023 e le categorie contigue della OWASP API Security Top 10 2023.	41
2.5	Relazioni fra API10:2023 e le categorie contigue della OWASP API Security Top 10 2023.	49

CAPITOLO 1

Introduzione

Alcune linee guida per scrivere la tesi: 1) Ogni immagine/tabella deve avere le seguenti caratteristiche: a) Una caption; b) Deve essere esplicitamente commentata all'interno del testo (ricordatevi che se vi riferite alla figura/tabella inserendo anche il corrispettivo numero e.g., Figura X mostra che... allora la prima lettera deve essere scritta in maiuscolo; E ricordatevi che latex mette a disposizione un comando per citare correttamente (ref) senza dover mettere a mano il numeretto corrispondente) c) Le immagini devono essere rifatte tramite PPT o similari e salvate in pdf prima di inserirle nel file latex, altrimenti rischiate di inserire immagini sgranate. 2) Ricordatevi sempre di citare le fonti e di rifrasare, altrimenti rischiate di incorrere nel reato di plagio (che può comportare anche la revoca del titolo accademico) e che citare non sminuisce il vostro lavoro, anzi, lo rafforza. 3) L'Abstract deve necessariamente contenere le seguenti informazioni a) Un breve riassunto della problematica; b) Una breve descrizione di ciò che è stato fatto; c) I risultati ottenuti.

Analisi dello stato del arte e contesto tecnologico

2.1 OWASP Cloud Security Top 10

Il portale AppSecMaster ha elaborato una propria classificazione dei dieci rischi più comuni e di maggiore impatto nei moderni ambienti cloud, ispirandosi ai principi promossi da OWASP in materia di sicurezza applicativa, che vanno dall'infrastruttura mal configurata alla scarsa visibilità delle risorse in esecuzione. Di seguito è riportata la lista delle principali problematiche individuate:

1. **Security Misconfiguration** — Bucket di storage lasciati con permessi troppo permissivi o istanze di calcolo distribuite senza le dovute restrizioni restano tra le cause più frequenti di violazione dei dati in ambienti cloud.
2. **Identity / Access Mismanagement** — Ruoli IAM assegnati con privilegi superiori a quelli effettivamente necessari, e mai rivisti dopo la configurazione iniziale, espongono i sistemi a un concreto rischio di escalation dei permessi.
3. **Improper Secrets/Key Management** - Se chiavi API e credenziali vengono salvate in chiaro o incorporate nel codice sorgente, la loro compromissione può aprire un accesso laterale a più servizi collegati.

4. **Insecure Network Segmentation** — Senza una reale separazione tra reti e ambienti diversi, un attaccante che viola un singolo componente può muoversi liberamente verso risorse più sensibili.
5. **Injection / Input Validation Flaws in Cloud Services** — Le classiche vulnerabilità di injection assumono forme nuove nei contesti cloud-native, colpendo funzioni serverless, code di messaggi e gateway API dove i controlli sugli input non sono sempre applicati in modo uniforme.
6. **Insecure Deserialization / API Exposure** — La gestione non sicura di dati serializzati, unita ad API interne prive di adeguati controlli di autenticazione o versionamento, costituisce un punto d'ingresso sfruttabile per eseguire codice non autorizzato.
7. **Vulnerable Third-Party/Container Dependencies** — Librerie open source e immagini container non aggiornate accumulano vulnerabilità note, soprattutto quando le stesse basi vengono riutilizzate tra più progetti senza verifiche periodiche.
8. **Improper Asset Inventory / Visibility** — Risorse cloud dimenticate o mai tracciate correttamente, incluse istanze abbandonate e servizi "ombra", sfuggono facilmente al controllo dei team di sicurezza.
9. **Insufficient Logging / Monitoring** — Log distribuiti su decine di servizi diversi e privi di un formato coerente rendono difficile individuare per tempo un'intrusione in corso.
10. **Unvalidated Redirects/Cloud Function Abuse** — Funzioni serverless eseguite con permessi troppo ampi, combinate a controlli insufficienti sui redirect, possono essere sfruttate per aggirare le restrizioni di accesso e ottenere un punto d'appoggio nascosto nell'infrastruttura.

Tra le problematiche precedentemente descritte , il presente lavoro si basa in particolare su Insecure Cloud Configurations e Insecure API Exposure per via della loro rilevanza diretta rispetto agli obiettivi del sistema sviluppato e alla loro

frequente ricorrenza negli scenari di autorizzazione tipici delle architetture basate su API.

2.1.1 Security Misconfiguration

Una misconfiguration, in ambito cloud, nasce da un errore nella configurazione di un servizio, di un prodotto o di una risorsa infrastrutturale: le impostazioni di sicurezza predefinite non vengono irrobustite a sufficienza, e questo lascia scoperto un fianco che con poco sforzo potrebbe essere protetto. I numeri parlano chiaro: il documento ufficiale OWASP riporta che il 90% delle applicazioni testate è risultato vulnerabile ad almeno una forma di misconfiguration [1]. Nella maggior parte dei casi non c'è nulla di sofisticato dietro: è una dimenticanza, un'impostazione lasciata di default, un permesso che nessuno ha mai rivisto. Ed è proprio questo che la distingue da una vulnerabilità nel codice applicativo — qui il problema non sta nella logica del software, ma in come le risorse sono state predisposte.

Le forme più comuni in cui una misconfiguration si manifesta sono, in fondo, poche e ricorrenti. Un bucket di storage lasciato accessibile pubblicamente, quando avrebbe dovuto restare privato, è probabilmente la più diffusa: basta una policy impostata male, magari copiata da un altro progetto senza troppa attenzione. Un container che gira con privilegi di root, invece, espone il sistema a un rischio diverso ma altrettanto concreto — se l'attaccante riesce a comprometterlo, si ritrova con permessi ben oltre quelli che gli servirebbero per fare danni. C'è poi il caso dei template Infrastructure-as-Code scritti senza le dovute verifiche: un file Terraform o CloudFormation che definisce una risorsa con permessi troppo ampi si porta dietro lo stesso errore ogni volta che viene riutilizzato, replicandolo su ambienti diversi senza che nessuno se ne accorga.

Un caso reale: il data breach Capital One Il caso Capital One del luglio 2019 resta probabilmente l'esempio più citato di questa categoria di rischio. Un bucket Amazon S3 lasciato con permessi troppo ampi permise a un'attaccante di sottrarre oltre 100 milioni di record clienti, tra dati anagrafici, indirizzi e informazioni finanziarie sensibili [2]. Nessuna falla nella piattaforma cloud era coinvolta: la responsabilità

ricadeva interamente sulla configurazione impostata dal cliente. Non a caso AWS, commentando l'accaduto, ribadì che *"Amazon S3 is secure by default"* [3], scaricando di fatto la colpa sulla gestione dei permessi lato utente. Sul piano economico il conto fu salato: 80 milioni di dollari di multa dall'Office of the Comptroller of the Currency, più 190 milioni versati per chiudere una class action. Capital One dimostra bene quanto una semplice disattenzione in fase di configurazione possa costare quanto, se non più di, un attacco costruito su un exploit tecnicamente elaborato.

Episodi come questo spiegano perché, negli ultimi anni, le organizzazioni abbiano iniziato ad affidarsi sempre meno alla revisione manuale delle configurazioni e sempre più a strumenti automatizzati capaci di individuare questo tipo di errori prima che vengano sfruttati. Una configurazione insicura, del resto, raramente viene introdotta di proposito: emerge quasi sempre da un template riutilizzato senza controllo, da un default mai cambiato, da una revisione saltata per mancanza di tempo — fattori che un controllo manuale sistematico difficilmente riesce a intercettare su larga scala e con continuità.

Il documento ufficiale OWASP illustra questa categoria di rischio attraverso quattro scenari esemplificativi [1]. Il primo riguarda applicazioni di esempio dimenticate su un server di produzione, spesso accompagnate da account amministrativi con credenziali mai modificate rispetto ai valori predefiniti. Il secondo descrive un server su cui la funzionalità di directory listing resta abilitata, consentendo a un attaccante di risalire fino al codice sorgente tramite reverse engineering. Il terzo riguarda messaggi di errore troppo dettagliati restituiti dall'application server, capaci di esporre informazioni sull'infrastruttura sottostante o versioni di componenti vulnerabili. Il quarto, infine, riguarda permessi di condivisione predefiniti di un cloud service provider lasciati aperti ad altri utenti su Internet, con conseguente esposizione di dati sensibili archiviati nello storage cloud.

Fra i quattro scenari, due ricadono nel perimetro del sistema presentato in questo lavoro e due ne restano fuori.

2.2 API Security Risks

Le API oggi fanno gran parte del lavoro che una volta faceva il sito web stesso: non restituiscono una pagina pensata per essere letta da una persona, ma dati grezzi destinati a un'altra app, a un altro servizio, a un sistema che poi decide cosa farne. Lo schema delle richieste è prevedibile, gli endpoint espongono operazioni precise su risorse precise. Per un attaccante questo significa meno lavoro di interpretazione: non deve capire un'interfaccia grafica, gli basta osservare come sono costruite le chiamate.

OWASP mantiene infatti, accanto al Top 10 per le applicazioni web, una classificazione separata dedicata alle API. Alcune categorie compaiono in entrambe le liste — la misconfiguration, il broken access control — ma buona parte dei rischi specifici delle API riguarda un livello che il Top 10 generale non copre: l'autorizzazione su singoli oggetti, proprietà, funzioni. Un'applicazione web tradizionale filtra cosa mostrare all'utente; una API deve filtrare cosa restituire nella risposta. La differenza sembra minima, ma è da lì che nasce buona parte delle vulnerabilità trattate in questa sezione. Le dieci categorie che compongono l'edizione 2023 riflettono proprio questa logica: non genericità, ma rischi specifici del modo in cui un'API espone dati e funzionalità. Di seguito la lista completa, in ordine di gravità secondo OWASP[4]:

1. **API1:2023 – Broken Object Level Authorization (BOLA)** — un endpoint accetta l'ID di una risorsa senza verificare che chi lo richiede abbia davvero il diritto di accedervi; basta cambiare un numero nell'URL per leggere dati altrui.
2. **API2:2023 – Broken Authentication** — meccanismi di autenticazione implementati male, token che non scadono mai, o password recuperabili senza controlli sufficienti.
3. **API3:2023 – Broken Object Property Level Authorization** — qui il problema non è accedere alla risorsa sbagliata, ma vedere o modificare campi che non dovrebbero essere esposti o scrivibili da quel particolare utente.
4. **API4:2023 – Unrestricted Resource Consumption** — assenza di limiti su chiamate, dimensione dei payload o uso di risorse computazionali, con conseguenze che vanno dal semplice rallentamento a un vero e proprio denial of service.

5. **API5:2023 – Broken Function Level Authorization** — un utente normale riesce a invocare funzioni riservate agli amministratori, semplicemente perché il controllo sui permessi manca a livello di endpoint o di metodo HTTP.
6. **API6:2023 – Unrestricted Access to Sensitive Business Flows** — la singola richiesta può essere anche legittima, ma se ripetuta migliaia di volte automatizza un abuso della logica di business (es. acquisto massivo di biglietti, creazione automatica di account).
7. **API7:2023 – Server Side Request Forgery (SSRF)** — l'API accetta un URL fornito dall'utente e lo interroga senza validarlo, permettendo di raggiungere risorse interne altrimenti non accessibili dall'esterno.
8. **API8:2023 – Security Misconfiguration** — header di sicurezza mancanti, CORS troppo permissivo, ambienti di debug ancora attivi in produzione: stessa famiglia di errori vista nel capitolo precedente, qui applicata allo strato applicativo.
9. **API9:2023 – Improper Inventory Management** — endpoint vecchi, versioni deprecate o ambienti di staging dimenticati online, mai rimossi né monitorati, restano comunque raggiungibili da chi li trova.
10. **API10:2023 – Unsafe Consumption of APIs** — la vulnerabilità non nasce nell'API stessa, ma nella fiducia cieca riposta nei dati restituiti da API di terze parti integrate nel sistema.

Tra le dieci categorie appena elencate, la prima merita un'attenzione diversa dalle altre. Non tanto per la posizione in classifica, quanto per quello che rappresenta: un controllo di accesso che manca proprio dove ci si aspetterebbe fosse più solido, cioè sulla singola risorsa richiesta. È il tipo di errore che si insinua facilmente in un endpoint scritto in fretta, e che allo stesso tempo può esporre l'intero set di dati di un sistema con una richiesta HTTP modificata di poco.

La sezione seguente approfondisce nel dettaglio Broken Object Level Authorization: come si manifesta, perché resta la vulnerabilità più diffusa tra quelle riscontrate nelle API reali, e possibili scenari di rimedio.

2.2.1 API1:2023 – Broken Object Level Authorization

Definizione e inquadramento concettuale La Broken Object Level Authorization (BOLA) si manifesta quando un'applicazione, pur autenticando correttamente l'utente che effettua una richiesta, non verifica se quello stesso utente abbia effettivamente il diritto di accedere all'oggetto specifico richiesto.

Un'interfaccia di programmazione applicativa espone tipicamente un insieme di risorse — utenti, ordini, documenti, transazioni — identificabili singolarmente attraverso un riferimento univoco. L'accesso a tali risorse presuppone due verifiche concettualmente distinte, spesso confuse nella pratica implementativa: l'autenticazione, che accerta l'identità del soggetto che effettua la richiesta, e l'autorizzazione, che stabilisce se quel soggetto, una volta identificato, abbia il diritto di operare su una specifica risorsa.

Un utente autenticato correttamente non è, per definizione, un utente autorizzato ad accedere a qualunque oggetto gestito dal sistema. Questa distinzione, apparentemente elementare, è proprio il punto in cui molte implementazioni falliscono: il possesso di credenziali valide viene talvolta considerato — implicitamente, nel disegno del codice — condizione sufficiente per l'accesso a una risorsa, quando in realtà rappresenta soltanto la premessa per una verifica ulteriore, relativa alla relazione specifica tra quell'utente e quell'oggetto.

Funzionamento delle API e ruolo degli identificatori di risorsa Le API REST, e in misura diversa quelle basate su GraphQL, espongono le risorse attraverso identificatori che compaiono nel percorso della richiesta, nei parametri di query, nel corpo del messaggio o negli header. Un endpoint come `GET /api/orders/482` restituisce, nella maggior parte delle implementazioni, l'oggetto ordine identificato dal valore 482, indipendentemente dal fatto che tale valore sia un intero progressivo, un UUID o un altro tipo di riferimento.

La gestione di una richiesta di questo tipo comporta, lato server, due passaggi distinti: la risoluzione dell'identificatore in un oggetto concreto, tramite interrogazione del database o di un servizio dati, e il controllo che l'utente autenticato abbia diritto di accesso a quell'oggetto specifico. Il primo passaggio è quasi sempre implementato

correttamente, poiché è indispensabile al funzionamento stesso dell'endpoint; il secondo, non avendo un effetto visibile quando manca, viene più facilmente trascurato o dato per scontato in fase di sviluppo.

Va osservato che l'endpoint, in questo scenario, non è di per sé vietato all'utente: la sua configurazione di ruoli e permessi lo autorizza a invocare quella funzione API. Questo distingue concettualmente BOLA da un'altra categoria della stessa famiglia, la Broken Function Level Authorization (BFLA), in cui è l'accesso alla funzione stessa a non essere autorizzato. In BOLA la funzione è legittima; il problema riguarda esclusivamente quale istanza dell'oggetto venga restituita o modificata.

Meccanismo dell'attacco Lo sfruttamento di una vulnerabilità BOLA non richiede, nella maggior parte dei casi, strumenti sofisticati né la compromissione di credenziali altrui. È sufficiente disporre di un account legittimo sul sistema bersaglio. Il percorso tipico dell'attacco segue una sequenza riconoscibile: l'utente accede al proprio account, richiede una propria risorsa attraverso l'interfaccia normale dell'applicazione e osserva, tramite gli strumenti di sviluppo del browser o un proxy di intercettazione, l'identificatore associato a quella risorsa nella richiesta sottostante.

A questo punto l'identificatore viene sostituito con un altro valore plausibile per lo stesso schema — un intero vicino, nel caso di sequenze numeriche, oppure un valore ottenuto per enumerazione più sistematica quando lo schema lo consente — e la richiesta viene reinviata con lo stesso token di autenticazione, valido e non modificato. Ad esempio:

```
GET /api/orders/483 HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_A>
```

dove 483 non è l'ordine appartenente al titolare del token, ma un identificatore vicino a quello osservato in precedenza durante il normale utilizzo dell'applicazione. La richiesta è, dal punto di vista del protocollo, del tutto regolare: cambia soltanto il valore del parametro. Se la risorsa restituita appartenga realmente a un altro utente dipende interamente da un unico fattore, approfondito nel paragrafo seguente: se

il server esegua o meno un confronto tra l'identità del chiamante e il proprietario effettivo dell'oggetto.

Esempio applicativo Si consideri il caso concreto della risorsa `/api/orders/{id}`, utilizzata per recuperare i dettagli di un ordine. Una richiesta legittima, effettuata dal proprietario dell'ordine 482, ha la forma:

```
GET /api/orders/482 HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_A>
```

e produce una risposta simile alla seguente:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "order_id": 482,
  "customer_id": 1041,
  "items": [...],
  "shipping_address": "...
}
```

La richiesta modificata introdotta nel paragrafo precedente, diretta all'ordine 483 e appartenente a `customer_id` 1078, viene gestita da un'implementazione vulnerabile allo stesso modo: l'identificatore viene estratto dal percorso, la tabella corrispondente viene interrogata, il risultato viene restituito. In nessun punto di questo flusso il valore di `customer_id` restituito viene confrontato con l'identità associata al token presente nell'header `Authorization`. La risposta, di conseguenza, è ancora un 200 OK, questa volta con i dati di un ordine che non appartiene al chiamante.

Il punto critico non è la struttura della richiesta, identica nei due casi, ma l'assenza di un confronto esplicito fra utente autenticato e risorsa richiesta. Un'implementazione corretta dovrebbe, prima di restituire l'oggetto, verificare che il `customer_id` dell'ordine coincida con l'identità del chiamante — o che quest'ultimo disponga di

un diritto di accesso esplicitamente concesso su quella risorsa — e restituire, in caso contrario, un errore di autorizzazione: tipicamente 403 `Forbidden`, oppure 404 `Not Found` quando si preferisce non rivelare l'esistenza della risorsa a chi non ne ha diritto.

Cause tecniche ricorrenti L'assenza di un controllo di autorizzazione a livello di oggetto è la causa immediata di ogni istanza di BOLA, ma raramente costituisce una scelta progettuale deliberata. Più spesso deriva da alcune condizioni che meritano di essere analizzate singolarmente.

La prima riguarda la sovrapposizione concettuale tra autenticazione e autorizzazione nella fase di implementazione. Un endpoint protetto da un middleware di autenticazione, che verifica la validità del token e ne estrae l'identità, può apparire sicuro a chi lo sviluppa, inducendo a trascurare la necessità di un controllo ulteriore specifico per l'oggetto acceduto. Il middleware risponde alla domanda "chi sta chiamando?", non a "questo chiamante ha diritto su questo oggetto?".

Una seconda causa è la fiducia implicita nei parametri forniti dal client. Alcune implementazioni derivano l'identità del proprietario di una risorsa dallo stesso identificatore fornito nella richiesta, oppure presuppongono che, poiché l'interfaccia utente dell'applicazione non espone mai identificatori altrui, un client "onesto" non li invierebbe mai. Questa assunzione ignora che un client HTTP può essere costruito o modificato arbitrariamente da chi effettua la richiesta, indipendentemente dall'interfaccia grafica ufficiale.

Un terzo fattore è rappresentato da controlli di autorizzazione implementati in modo incompleto o incoerente tra i diversi endpoint di una stessa API. È relativamente comune riscontrare verifiche corrette sull'operazione di cancellazione di una risorsa e verifiche assenti sull'operazione di lettura o aggiornamento della medesima risorsa, poiché ciascun endpoint viene sviluppato e sottoposto a revisione in momenti diversi, con criteri non sempre uniformi. La crescita organica di un'API nel tempo, con l'aggiunta progressiva di nuovi endpoint da parte di sviluppatori diversi, tende ad amplificare questa disomogeneità.

Una causa più strutturale riguarda infine la gestione delle relazioni tra utenti e risorse nel modello dati. Quando tale relazione non è rappresentata in modo esplicito

e interrogabile — ad esempio tramite una chiave esterna diretta tra la tabella degli utenti e quella delle risorse, o tramite un livello di autorizzazione centralizzato — ogni endpoint deve reimplementare autonomamente la logica di verifica, e questo aumenta la probabilità che qualche percorso venga trascurato.

Impatto e conseguenze L’impatto di una vulnerabilità BOLA dipende dalla natura della risorsa esposta, ma incide tipicamente su più proprietà di sicurezza contemporaneamente. Sul piano della riservatezza, l’accesso non autorizzato a un oggetto consente la lettura di dati che dovrebbero rimanere riservati al proprietario legittimo: informazioni personali, dettagli finanziari, contenuti sanitari, comunicazioni private, a seconda del dominio applicativo.

Quando l’endpoint vulnerabile consente anche operazioni di scrittura, l’impatto si estende all’integrità delle risorse: un attaccante può modificare dati appartenenti ad altri utenti, alterare lo stato di un ordine, cambiare le impostazioni di un account che non gli appartiene. Se l’operazione esposta è una cancellazione, la conseguenza diventa la perdita di disponibilità della risorsa per il legittimo proprietario.

Un aspetto che distingue BOLA da molte altre classi di vulnerabilità è la sua natura sistematicamente ripetibile: una volta individuato un endpoint privo del controllo appropriato, lo stesso schema di attacco può essere applicato a un numero elevato di identificatori, trasformando quella che sembra una singola falla puntuale in un canale di accesso a porzioni estese della base dati. Diversi incidenti di sicurezza documentati pubblicamente, inclusi casi in cui decine di milioni di record sono stati esposti, hanno questa origine comune.

Il rischio assume connotati particolari negli ambienti multi-tenant e cloud, dove più clienti condividono la stessa infrastruttura applicativa e la separazione tra i rispettivi dati dipende interamente dalla correttezza della logica di autorizzazione, non da una separazione fisica delle risorse. In un simile contesto, una singola vulnerabilità BOLA non compromette soltanto i dati di un utente, ma può erodere il confine logico tra tenant distinti, con implicazioni che vanno oltre il singolo incidente e toccano la fiducia complessiva riposta nell’infrastruttura condivisa.

Strategie di mitigazione La prevenzione di BOLA richiede che ogni accesso a una risorsa identificata da un parametro fornito dal client sia accompagnato da una verifica esplicita del diritto dell'utente su quello specifico oggetto, eseguita interamente lato server. Nessuna informazione proveniente dal client, incluso l'identificatore stesso della risorsa, può essere considerata prova di autorizzazione: l'identificatore indica quale oggetto viene richiesto, non chi ha diritto di accedervi.

Un principio applicabile in modo trasversale è quello del *least privilege*, secondo cui un soggetto dovrebbe disporre soltanto dei permessi strettamente necessari a svolgere le proprie funzioni, senza margini di accesso più ampi per comodità implementativa. Applicato al livello oggetto, questo si traduce nella necessità di associare esplicitamente ogni utente, o ruolo, alle risorse su cui ha effettivamente diritto di operare, evitando che l'accesso derivi da assunzioni implicite sulla struttura delle richieste.

Dal punto di vista architetturale, è preferibile centralizzare la logica di autorizzazione in un livello comune — un modulo, un middleware o un servizio dedicato — piuttosto che reimplementarla in modo indipendente in ciascun endpoint. Questa scelta non elimina la necessità di applicare il controllo in ogni punto di accesso, ma riduce la probabilità che errori o dimenticanze locali producano percorsi non protetti, e semplifica la verifica di coerenza tra le diverse funzioni esposte dall'API.

La coerenza dei controlli tra operazioni di lettura, scrittura e cancellazione sulla stessa risorsa merita infine attenzione specifica: la protezione di un'operazione non implica automaticamente la protezione delle altre, ed è proprio in questa asimmetria che si annidano molte delle istanze reali di BOLA documentate in letteratura.

Nei paragrafi seguenti vengono trattate più sinteticamente le restanti vulnerabilità della OWASP API Security Top 10, a partire da API2:2023.

2.2.2 API2:2023 – Broken Authentication

Definizione e collocazione Broken Authentication occupa la seconda posizione della classificazione OWASP e si colloca concettualmente a monte di BOLA: mentre quest'ultima riguarda il controllo sull'accesso a uno specifico dato o risorsa, Broken Authentication compromette la capacità stessa del sistema di stabilire con certezza l'identità di chi effettua la richiesta. Un fallimento in questa fase rende vano qualsiasi controllo successivo, poiché un meccanismo di autorizzazione perde ogni efficacia se l'identità su cui si fonda può essere falsificata o aggirata [5]. A differenza di altre categorie più circoscritte, questa voce raggruppa un insieme eterogeneo di debolezze nei processi di verifica dell'identità: dalla gestione poco attenta dei token di sessione all'assenza di vincoli sul numero di tentativi di accesso consentiti. Un esempio comune riguarda i messaggi di risposta del server: se il sistema segnala in modo esplicito se a essere errato sia lo username oppure la password, fornisce involontariamente all'attaccante una conferma sull'esistenza di un dato account. Parallelamente, l'assenza di limitazioni sulla frequenza delle richieste lascia l'interfaccia di login esposta ad attacchi automatizzati volti a indovinare le credenziali su larga scala.

Cause e meccanismi tecnici tipici Spesso queste problematiche nascono dal tentativo di implementare soluzioni di autenticazione proprietarie o non standard. Come evidenziato dalla documentazione OWASP, strumenti come le API key o protocolli di autorizzazione come OAuth vengono talvolta impiegati impropriamente per identificare gli utenti in scenari per cui non sono stati progettati, creando falle strutturali [5]. Un ulteriore fattore critico è la mancanza di controlli sul volume dei tentativi di accesso. Endpoint di autenticazione e flussi di recupero password privi di rate limiting risultano vulnerabili ad attacchi in cui elenchi di credenziali già compromesse altrove vengono testati a raffica, sfruttando la tendenza diffusa a riutilizzare le medesime password su più servizi [6]. Infine, la gestione dei token di sessione (come i JSON Web Token, o JWT) introduce ulteriori insidie: errori nella convalida della firma crittografica, chiavi di cifratura deboli, mancato controllo dei tempi di scadenza o assenza di meccanismi di revoca immediata permettono a un attaccante di forgiare token validi o di continuare a utilizzare credenziali che dovrebbero essere

già decadute [7].

Impatto essenziale L'impatto di Broken Authentication è per sua natura sistemico: assumendo il controllo dell'identità di un utente, l'attaccante ne eredita tutti i privilegi operativi senza che il backend possa distinguere le azioni legittime da quelle malevole [5]. Ciò può tradursi non solo nell'accesso a dati riservati, ma anche nella manomissione dei parametri di sicurezza dell'account (come la modifica dell'email di recupero), trasformando un'intrusione temporanea nella perdita definitiva del profilo.

Mitigazione La contromisura principale consiste nell'affidarsi a standard industriali consolidati e a librerie verificate, evitando soluzioni "fai-da-te" per la gestione di token e password [8]. A livello applicativo e infrastrutturale è indispensabile introdurre politiche di rate limiting dedicate e restrittive sugli endpoint di login, adottare l'autenticazione a più fattori (MFA) dove possibile e imporre una nuova conferma delle credenziali prima di eseguire modifiche critiche sul profilo utente [9].

Rilevabilità Sul piano della rilevabilità, l'analisi automatica si divide principalmente in due direttrici a seconda della natura del difetto:

1. **Analisi dei Token e della Crittografia:** Per quanto riguarda la gestione dei JWT e dei formati di sessione, le vulnerabilità sono per lo più legate a regole deterministiche e ben definite (come la mancata verifica della firma o l'uso di algoritmi insicuri). La ricerca e gli strumenti di settore riescono quindi a scovare queste falle in modo efficace, sia analizzando staticamente il codice sorgente per intercettare configurazioni errate delle librerie crittografiche, sia generando dinamicamente token manipolati per verificare se il backend li accetti erroneamente [10, 11].
2. **Flussi di Autenticazione e Abusi Comportamentali:** Al contrario, individuare l'assenza di rate limiting o la vulnerabilità a tentativi di accesso massivi richiede un approccio basato sul monitoraggio a runtime. Più che da un'ispezione sintattica del codice, queste carenze emergono dall'analisi dinamica del traffico

o dall'osservazione dei registri di sistema (log), rilevando picchi anomali di richieste fallite concentrate in brevi finestre temporali [6].

2.2.3 API3:2023 – Broken Object Property Level Authorization

Definizione e collocazione La *Broken Object Property Level Authorization* (BOPLA) identifica una classe di vulnerabilità nella quale i controlli di autorizzazione applicati alle API non considerano adeguatamente le singole proprietà degli oggetti gestiti dall'applicazione. Il problema non riguarda quindi soltanto la possibilità per un utente di accedere a un determinato oggetto, ma anche quali attributi di quell'oggetto possa effettivamente leggere o modificare.

La distinzione rispetto alla *Broken Object Level Authorization* (BOLA) è rilevante. Nel caso di BOLA, il controllo di autorizzazione fallisce a livello dell'oggetto: un utente riesce, per esempio, ad accedere a una risorsa appartenente a un altro utente manipolando l'identificativo della risorsa. Nel caso di BOPLA, invece, l'utente può avere legittimamente accesso all'oggetto, ma ottenere o modificare proprietà che non rientrano nei suoi privilegi. L'oggetto, quindi, non costituisce necessariamente la risorsa a cui viene negato l'accesso; è una sua specifica proprietà a essere esposta o manipolabile senza un'adeguata autorizzazione.

Il rischio è particolarmente rilevante nelle API REST, dove è frequente che un endpoint restituisca direttamente la rappresentazione serializzata di un oggetto. Una struttura interna contenente numerosi attributi può così diventare, senza una selezione esplicita, la struttura della risposta esposta al client.

Cause e meccanismi tecnici tipici Una delle cause più comuni è la sovrapposizione tra il modello utilizzato internamente dall'applicazione e quello esposto attraverso l'API. Un oggetto memorizzato nel database può contenere, ad esempio, proprietà quali `id`, `name`, `email`, `role` e `internalStatus`. Non tutte queste proprietà hanno però la stessa finalità né devono necessariamente essere accessibili allo stesso insieme di utenti.

Un'implementazione che serializza automaticamente l'intero oggetto può produrre una risposta contenente anche attributi che il client non dovrebbe conoscere. In

questo caso il problema non deriva necessariamente da un accesso non autorizzato all'oggetto, ma dal fatto che l'API non applica una politica distinta alle proprietà che lo compongono. L'interfaccia grafica potrebbe nascondere determinati campi, ma ciò non costituisce una misura di sicurezza: il contenuto della risposta HTTP rimane osservabile dal client e può essere analizzato direttamente.

Un secondo meccanismo riguarda le operazioni di scrittura. Framework e librerie possono consentire il *binding* automatico dei parametri ricevuti dal client verso gli attributi di un oggetto applicativo. Se il server accetta un payload come:

```
{
  "name": "Mario",
  "email": "[mario@example.com] (mailto:mario@example.com) "
}
```

un attaccante può provare ad aggiungere proprietà che non erano previste dal normale flusso applicativo:

```
{
  "name": "Mario",
  "email": "[mario@example.com] (mailto:mario@example.com) ",
  "role": "admin"
}
```

Se il campo `role` viene accettato e aggiornato senza un controllo specifico, l'utente può modificare una proprietà interna che dovrebbe essere gestita esclusivamente dal sistema. Lo stesso principio può riguardare attributi quali `isAdmin`, `approved`, `price` o `ownerId`, a seconda del modello utilizzato dall'applicazione.

Il problema può quindi manifestarsi in due direzioni differenti: lettura non autorizzata di proprietà e modifica non autorizzata di proprietà. Nel primo caso l'attaccante osserva informazioni che non dovrebbe ricevere; nel secondo riesce a influenzare direttamente lo stato dell'applicazione attraverso attributi che non dovrebbero essere controllabili dal client.

Impatto essenziale L'impatto di una vulnerabilità BOPLA dipende principalmente dal tipo di proprietà coinvolta e dalle funzioni che la utilizzano. La semplice esposizione di un attributo può comportare una violazione della riservatezza, mentre la possibilità di modificarlo può compromettere l'integrità dei dati o il funzionamento dell'applicazione.

Nel caso di proprietà contenenti informazioni personali, finanziarie o relative all'organizzazione interna del sistema, l'esposizione può determinare una divulgazione di dati a soggetti non autorizzati. La possibilità di modificare determinate proprietà può invece portare alla perdita o alla corruzione dei dati. In situazioni più critiche, una proprietà utilizzata nei meccanismi di autorizzazione o nella gestione dello stato dell'account può essere sfruttata per ottenere un'escalation dei privilegi o contribuire alla compromissione dell'account.

La gravità della vulnerabilità dipende quindi dal significato assunto dalla proprietà all'interno dell'applicazione

Mitigazione La mitigazione di BOPLA richiede che l'autorizzazione venga applicata non soltanto all'endpoint o all'oggetto, ma anche alle proprietà che possono essere restituite o modificate. La prima misura consiste quindi nel definire esplicitamente quali attributi ogni endpoint può esporre e quali possono essere accettati in input.

Per le risposte, è preferibile evitare l'utilizzo di serializzazione quando questi comportano la restituzione automatica dell'intero oggetto. È invece opportuno selezionare esplicitamente le proprietà necessarie alla specifica operazione.

Per le operazioni di scrittura è necessario limitare esplicitamente gli attributi modificabili dal client. Una tecnica comune consiste nell'utilizzare una *whitelist* delle proprietà ammesse, evitando che il payload venga associato automaticamente a tutti gli attributi dell'oggetto.

Rilevabilità La rilevazione di BOPLA si articola in due verifiche distinte, corrispondenti alle due direzioni in cui una proprietà può attraversare il confine fra client e server.

La prima riguarda le proprietà esposte in risposta a una richiesta legittima, e si riduce al confronto fra gli attributi effettivamente restituiti e quelli previsti per il

ruolo o il contesto dell'utente. È la verifica più diretta, perché l'evidenza è contenuta nella risposta stessa.

La seconda riguarda le proprietà accettate in ingresso, ed è più onerosa perché il server non le dichiara. Individuarle richiede di modificare il payload delle richieste, eventualmente ricorrendo a tecniche di fuzzing per scoprire attributi non documentati, e di verificare poi se il server ne recepisca effettivamente il valore.

Le due verifiche non sono sostituibili l'una con l'altra: un'API può accettare in input una proprietà sensibile che non restituisce mai, oppure restituire dati riservati senza consentirne la modifica. Un'analisi che si limitasse a una sola direzione lascerebbe scoperto metà del problema.

2.2.4 API4:2023 – Unrestricted Resource Consumption

Definizione e collocazione Ogni richiesta servita da un'API consuma risorse: banda, CPU, memoria, storage e, con frequenza crescente, la categoria API4:2023 raccoglie i casi in cui almeno uno dei limiti che dovrebbero governare tale consumo è assente o dimensionato in modo inadeguato [12]. Il documento OWASP elenca i limiti attesi: timeout di esecuzione, memoria allocabile, numero di descrittori di file e di processi, insieme a molte altre categorie. [12].

La collocazione nella Top 10 richiede un chiarimento ulteriore, perché il consumo di risorse non è un difetto di una specifica funzione ma una proprietà trasversale di qualunque endpoint, e i limiti che lo governano sono la difesa in profondità di altre categorie. Quattro relazioni contano più delle altre.

Con API2:2023 il rapporto è di specializzazione. Broken Authentication chiede che gli endpoint di autenticazione e di recupero credenziali siano protetti da meccanismi anti-brute-force più severi del rate limiting ordinario applicato al resto dell'API [5]. Il meccanismo di aggiramento è però lo stesso che API4 elenca fra i limiti mancanti, ossia il numero di operazioni ammesse in una singola richiesta come nel batching GraphQL [12]. A cambiare è il bene protetto, e i due scenari illustrativi lo mostrano bene: in API2 il batch aggrega tentativi di login per superare un limite di tre richieste al minuto e arrivare alla credenziale [5], in API4 aggrega caricamenti di immagini fino a esaurire la memoria del server [12].

Con API6:2023 il vettore resta l'automazione delle richieste e cambia l'esito. OWASP colloca il danno sul piano del business e scrive, nella descrizione degli impatti, che un impatto tecnico in generale non è atteso [13]. Lo scenario di apertura descrive l'acquisto automatico dell'intero stock di una console distribuito su indirizzi IP e località diverse [13]: una costruzione che aggira per definizione il throttling per IP, cioè la forma di limitazione più semplice da implementare e anche la più fragile.

Con API10:2023 la sovrapposizione è nel testo stesso. Fra gli indicatori di Unsafe Consumption of APIs compaiono l'assenza di un limite alle risorse impiegate per elaborare le risposte dei servizi terzi e l'assenza di timeout nelle interazioni con essi [14], due condizioni che API4 annovera già fra i limiti la cui mancanza rende un'API vulnerabile [12]. A distinguerle è la direzione del consumo: API4 guarda alle risorse che un client sottrae al server, API10 a quelle che un fornitore a monte può far spendere al server.

Con API9:2023 il problema non è l'assenza del limite ma la sua copertura parziale. Lo scenario di Improper Inventory Management descrive un rate limiting collocato non nel codice dell'API ma in un componente separato fra client e API ufficiale, e un host beta che espone la stessa API senza quel componente: da lì è stato possibile forzare per tentativi il token di reset a sei cifre e cambiare la password di qualunque utente [15]. OWASP ne ricava due indicazioni operative, documentare il rate limiting fra gli aspetti del contratto API ed estendere le misure di protezione esterne a tutte le versioni esposte e non solo a quella in produzione [15].

Categoria	Punto di contatto con API4	Elemento distintivo
API2	Gli endpoint di autenticazione e recupero credenziali richiedono limiti più severi di quelli ordinari; il batching GraphQL aggira il rate limiting in entrambe le categorie	Bene protetto: la segretezza della credenziale (CWE-307), dove API4 tutela disponibilità e costo (CWE-770, CWE-799)
API6	Automazione di richieste di per sé legittime; distribuendo l'attacco si elude il throttling per indirizzo IP	Danno collocato sul piano del business, con impatto tecnico in generale non atteso; contromisure comportamentali
API9	Il limite può risiedere in un componente esterno al codice dell'API e non coprire tutti gli host esposti	Difetto di inventario e documentazione: il codice resta corretto mentre la superficie esposta rimane scoperta
API10	Assenza di timeout e di limiti sulle risorse impiegate per elaborare le risposte di servizi terzi	Direzione del consumo: le risorse che un fornitore a monte fa spendere al server

Tabella 2.1: Relazioni fra API4:2023 e le categorie contigue della OWASP API Security Top 10 2023.

Cause e meccanismi tecnici tipici La radice comune è un'asimmetria: produrre una richiesta costa al client una quantità trascurabile di risorse, mentre servirla può costare al server una quantità che cresce in funzione di parametri controllati dal client stesso. Ogni volta che questa funzione di costo non è limitata superiormente, l'API è esposta. Conviene distinguere quattro meccanismi, perché ciascuno richiede una difesa diversa e presenta un profilo di rilevabilità distinto, e poi chiedersi perché, in pratica, i limiti vengano omessi.

Il primo meccanismo è il volume puro: l'assenza di un limite alla frequenza delle richieste per client, per utenza o per endpoint. OWASP osserva che lo sfruttamento non richiede competenze particolari, poiché richieste concorrenti possono essere generate da una singola macchina o da risorse cloud noleggiate, e che la maggior parte

degli strumenti automatici disponibili mira proprio a saturare il tasso di servizio [12]. La forma più insidiosa non è però l'assenza totale del limite, ma la sua presenza in una sola dimensione: un limite per indirizzo IP è inefficace contro un attacco distribuito, un limite per endpoint è inefficace contro il batching, un limite per utente autenticato non copre gli endpoint anonimi, che sono tipicamente quelli di registrazione e recupero credenziali.

Il secondo è il parametro che governa la dimensione della risposta. Un endpoint paginato che accetta `limit=100000`, o che non impone alcun tetto, trasforma una singola richiesta in una scansione completa della tabella sottostante. Non a caso OWASP colloca fra le contromisure la validazione lato server dei parametri che controllano il numero di record restituiti [12]. Il caso si intreccia con API3:2023, già trattato: la stessa disattenzione verso la forma della richiesta produce l'esposizione di proprietà, qui esplosione del carico, e un endpoint che soffre di entrambe consente a un attaccante di estrarre l'intero insieme di dati sensibili con una manciata di chiamate.

Il terzo meccanismo è la moltiplicazione di operazioni in una sola richiesta. Lo scenario GraphQL riportato da OWASP è istruttivo: un'API applica rate limiting all'endpoint `/graphql` e controlla la dimensione delle immagini caricate, ma non limita il numero di mutazioni aggregate in un unico batch; l'attaccante invia centinaia di `uploadPic` in una richiesta sola, ciascuna delle quali innesca la generazione di miniature, e satura la memoria del server aggirando entrambe le protezioni [12]. La letteratura ha formalizzato il fenomeno: Cha et al. mostrano che una query GraphQL nel caso peggiore impone al server una quantità di lavoro esponenziale e ricordano, richiamando lo studio empirico precedente dello stesso gruppo, che la maggioranza degli schemi pubblici espone il fornitore a query ad alto costo [16]. Il meccanismo non è esclusivo di GraphQL: qualunque endpoint REST che accetti un array di operazioni, come un'importazione massiva o un aggiornamento in blocco, presenta lo stesso profilo se non limita la cardinalità dell'array.

Il quarto meccanismo non dipende dal numero di richieste ma dalla complessità algoritmica dell'elaborazione di un singolo input. È la famiglia degli attacchi di complessità algoritmica descritta da Crosby e Wallach [17], di cui il ReDoS (*Regular expression Denial of Service*) è l'istanza più diffusa nel contesto delle API: un'espres-

sione regolare soggetta a backtracking catastrofico, applicata a un input costruito ad arte, può bloccare un thread per minuti. Staicu e Pradel hanno documentato la gravità del fenomeno nei server basati su JavaScript, la cui esecuzione a thread singolo rende un'unica richiesta sufficiente a congelare l'intero processo [18]. Nella stessa famiglia rientrano i timeout mancanti sulle chiamate in uscita verso altri servizi, che consentono a un fornitore lento di trattenere indefinitamente connessioni e worker.

A questi si aggiunge la causa che l'edizione 2023 porta in primo piano: l'assenza di soglie di spesa verso i fornitori esterni. Qui la risorsa consumata non è computazionale ma economica, e il limite mancante non risiede nel codice ma nel contratto con il provider [12].

Resta da spiegare perché limiti così semplici da enunciare vengano omessi con la frequenza che OWASP classifica come diffusa. Tre ragioni ricorrono. La prima è la diffusione della responsabilità: un limite può essere applicato dal framework, dal reverse proxy, dal gateway, dall'orchestratore o dal fornitore cloud, e quando ogni livello presume che lo faccia un altro il risultato è che nessuno lo fa. La seconda è che il difetto non produce errori: un endpoint senza limiti funziona perfettamente sotto carico normale e supera ogni test funzionale, sicché nulla nel ciclo di sviluppo lo segnala. La terza è l'elasticità dell'infrastruttura, che nasconde il sintomo trasformando l'esaurimento in scalabilità automatica e quindi in costo; il problema diventa visibile alla fattura successiva, non nel monitoraggio.

Impatto essenziale L'esito più immediato è il denial of service per esaurimento. CWE-770 lo enumera come conseguenza principale sulla disponibilità, per CPU, memoria o altre risorse, notando che l'attaccante può ottenerlo sia con molte richieste rapide sia inducendo l'uso di risorse più grandi del necessario [19]. Diversamente da un attacco volumetrico di rete, il punto di rottura non è la banda ma la logica applicativa, e la quantità di traffico necessaria può essere di ordini di grandezza inferiore: una singola richiesta ReDoS o un solo batch GraphQL sono sufficienti, e nessuna difesa perimetrale basata sul volume li riconosce come anomali.

L'edizione 2023 insiste però su una seconda dimensione, quella economica, e le riconosce pari dignità. Due dei tre scenari illustrativi non causano alcuna interruzione del servizio. Nel primo, un flusso di recupero password che invia un codice via SMS

attraverso un fornitore che tariffa 0,05 dollari a messaggio viene invocato decine di migliaia di volte da uno script, con perdite di migliaia di dollari in pochi minuti. Nel terzo, un file distribuito tramite API supera la soglia di 15 GB oltre la quale la cache non opera e, in assenza di allarmi sui costi, la fattura mensile per lo storage passa da circa 13 a circa 8 000 dollari [12]. Il tratto comune è che l'infrastruttura elastica converte l'esaurimento in spesa: il servizio resta disponibile, ma a un costo che nessuno ha autorizzato. OWASP nota anche che l'attaccante, pur privo di visibilità diretta sui costi, può inferirli dal modello di prezzo del provider [12].

Un terzo effetto, meno citato, è la degradazione progressiva: prima del collasso, la latenza cresce per tutti gli utenti legittimi e, nelle architetture a microservizi, l'esaurimento di un componente si propaga a monte attraverso code e tentativi ripetuti. In questo senso l'impatto di API4 è raramente confinato all'endpoint vulnerabile: un pool di connessioni al database saturato da una query di paginazione senza tetto rende indisponibili anche gli endpoint correttamente limitati che condividono lo stesso pool.

Mitigazione Le contromisure si dispongono su livelli distinti e nessuno, da solo, è sufficiente. Al livello più basso, OWASP raccomanda di affidare i limiti fisici (memoria, CPU, numero di processi e descrittori, riavvii) a piattaforme che li rendono dichiarativi, come container e funzioni serverless [12]. Si tratta di una difesa di ultima istanza: non impedisce l'abuso, lo confina.

Al livello dell'ingresso dei dati, la regola è dichiarare e applicare una dimensione massima per ogni parametro e payload: lunghezza delle stringhe, cardinalità degli array, dimensione dei file e, in particolare, il valore dei parametri che controllano il numero di record restituiti [12]. CWE-770 inquadra la stessa misura come validazione dell'input per lista di valori ammessi, con l'avvertenza che copre solo i casi in cui l'input influenza dimensione o frequenza delle allocazioni [19]. Il contratto OpenAPI è il luogo naturale per rendere questi vincoli espliciti attraverso `maxLength`, `maxItems` e `maximum` sui parametri di paginazione. Un contratto che dichiara i limiti ha un valore che va oltre la documentazione: rende i limiti verificabili da uno strumento, tema su cui si tornerà.

Al livello delle interazioni, il rate limiting resta la contromisura di riferimento,

con due precisazioni ricorrenti nel documento OWASP: va tarato per endpoint in base alle esigenze di business, e va affiancato da un limite sul numero di esecuzioni di una singola operazione per utente (validazione di un OTP, richiesta di reset), che il solo conteggio delle chiamate all'endpoint non cattura [12]. La seconda precisazione è quella che lo scenario GraphQL rende evidente. Per gli endpoint di autenticazione il requisito si irrigidisce: OWASP chiede meccanismi anti-brute-force più severi del rate limiting ordinario e il trattamento degli endpoint di recupero credenziali alla stregua di quelli di login [5], e le linee guida NIST sull'autenticazione digitale prescrivono al verificatore controlli di throttling contro gli attacchi di indovinamento in linea [8]. Sul piano protocollare, HTTP fornisce il codice di stato 429 `Too Many Requests`, definito in RFC 6585 [20], e l'IETF sta standardizzando i campi `RateLimit` e `RateLimit-Policy` con cui il server comunica al client quota residua e finestra temporale, consentendo una regolazione cooperativa del traffico [21]; al momento della stesura il documento è ancora una Internet-Draft. NIST tratta il throttling tra le strategie di sicurezza per le applicazioni a microservizi, collocandolo nel gateway o nella mesh piuttosto che nel singolo servizio [22]. Quest'ultima indicazione va letta insieme allo scenario dell'host beta già richiamato: collocare il limite nel gateway è corretto purché ogni host che espone l'API stia dietro quel gateway, e il limite applicato nel codice, per quanto meno elegante, ha il vantaggio di viaggiare con l'applicazione in ogni ambiente in cui viene distribuita.

Per la famiglia algoritmica la mitigazione è specifica: timeout su ogni operazione potenzialmente lunga (chiamate in uscita, elaborazione di immagini, matching di espressioni regolari) e, per il ReDoS, l'adozione di motori regex a tempo lineare o la riscrittura delle espressioni vulnerabili [18]. Per GraphQL, l'analisi statica del costo di una query prima della sua esecuzione, come quella proposta da Cha et al., permette di rifiutare richieste il cui limite superiore di complessità eccede una soglia [16].

Per la dimensione economica, infine, OWASP prescrive soglie di spesa su tutti i fornitori integrati e, dove il provider non le supporta, almeno allarmi di fatturazione [12]. È una misura amministrativa più che tecnica, ma è l'unica che intercetta lo scenario del file da 18 GB.

CWE-770 aggiunge una considerazione di progetto che vale come avvertenza: il throttling su base identitaria può a sua volta diventare un vettore, perché un

attaccante che impersona un utente legittimo può farlo bloccare; il throttling uniforme evita questo problema ma non risolve l'attacco, si limita a renderlo più costoso [19]. Il compromesso fra le due strategie va deciso caso per caso, e la risposta cambia fra un endpoint di login, dove il blocco dell'account è un rischio concreto di negazione del servizio mirata, e un endpoint di ricerca, dove il throttling per utente non danneggia terzi.

Rilevabilità Le due fonti primarie sembrano contraddirsi. OWASP classifica la rilevabilità come facile, motivando che richieste costruite con parametri che controllano il numero di risorse restituite, seguite dall'analisi di stato, tempo e lunghezza della risposta, sono normalmente sufficienti a identificare il problema, e che lo stesso vale per le operazioni batch [12]. CWE-770, nella sezione sui metodi di rilevamento, giudica invece l'analisi statica automatica di utilità limitata per le allocazioni senza limite, con eccezioni circoscritte (mancato rilascio di risorse di sistema, argomenti non verificati alle allocazioni di memoria, file di configurazione privi di un valore massimo), e la ritiene inadatta per le risorse definite dall'applicazione, come una politica che consenta a un utente un numero massimo di operazioni al giorno [19]. La contraddizione è apparente: OWASP descrive la prospettiva del tester in black box, CWE quella dello strumento che analizza il codice. Le due prospettive vanno trattate separatamente.

Sul versante dinamico l'evidenza è consolidata. CWE-770 riconosce che le tecniche automatiche basate sulla generazione di un numero elevato di richieste in un intervallo breve sono efficaci nel produrre gli effetti collaterali dell'allocazione incontrollata, pur richiedendo interpretazione manuale dei risultati, e che il fuzzing può rilevare la debolezza in modo opportunistico, specialmente quando l'allocazione dipende da input numerici [19]. Il lavoro di Antunes, Neves e Veríssimo, citato dalla stessa voce CWE, è fra i primi a proporre un approccio sistematico di rilevamento e predizione delle vulnerabilità da esaurimento nei server di rete tramite iniezione di carico [23]. La metodologia OWASP, ridotta all'essenziale, è una forma di test differenziale: si confrontano le risposte a richieste con parametri di dimensione crescente, e un tempo o una lunghezza che crescono senza saturare rivelano l'assenza del limite. Il test dinamico ha un vantaggio decisivo rispetto a ogni analisi del codice: osserva il

sistema nel suo insieme, e quindi vede il limite ovunque sia applicato. Ha però un limite simmetrico: osserva solo l'host che interroga, e un host beta senza gateway resta invisibile finché non viene inventariato.

Sul versante statico la letteratura si concentra su sottoclassi ben delimitate. Per il ReDoS esiste un filone maturo: Staicu e Pradel combinano identificazione statica di espressioni regolari vulnerabili e verifica dinamica su siti reali, mostrando che il problema è diffuso e sfruttabile in produzione [18]. Per GraphQL, l'analisi di Cha et al. calcola in tempo lineare un limite superiore del costo di una query senza eseguirla, ed è stata validata su API commerciali confrontando la complessità predetta con quella osservata nelle risposte [16]; è a tutti gli effetti un rilevatore statico, sebbene operi sulla query in arrivo e non sul codice del server.

Ciò che manca, per quanto è stato possibile verificare, è un corpo di lavori sottoposti a revisione paritaria dedicato al rilevamento statico dell'assenza di rate limiting, di timeout o di limiti di paginazione nel codice sorgente di API REST. Le pratiche esistenti in questo spazio provengono dagli strumenti e non dalla letteratura, e si lasciano raggruppare in tre famiglie di segnali. La prima opera sul codice: regole di pattern matching che segnalano chiamate HTTP in uscita prive di timeout, applicazioni web che non registrano alcun middleware di limitazione, parametri di paginazione letti dalla richiesta e passati alla query senza un tetto, espressioni regolari con struttura vulnerabile applicate a input esterni. La seconda opera sui manifest infrastrutturali: presenza di limiti di risorse per i container, dimensione massima del corpo della richiesta nel reverse proxy, piani di throttling sui gateway. La terza opera sul contratto OpenAPI: schemi privi di `maxLength`, `maxItems` o `maximum`, endpoint che dichiarano parametri di paginazione senza vincoli, assenza di risposte 429 documentate. L'osservazione di CWE-770 sui file di configurazione privi di un massimo è, di fatto, la sola indicazione proveniente da una fonte primaria che legittima la seconda e la terza direzione [19].

Il quadro suggerisce una conclusione precisa sulla natura del segnale statico, che distingue API4 dalle categorie trattate finora. Per una BOLA o un difetto di autenticazione, l'assenza del controllo nel codice del gestore è la vulnerabilità, perché il controllo non può che stare lì. Per API4 l'analisi del codice non può affermare la presenza della vulnerabilità, perché il limite potrebbe essere applicato altrove: nel

gateway, nel reverse proxy, nella piattaforma di orchestrazione. Può però affermare con buona confidenza l'assenza di un controllo in un dato livello, e questa assenza diventa un finding solo quando è confermata sugli altri livelli o da una sonda dinamica. Lo scenario di API9 mostra anche il caso opposto: la presenza del limite in un livello non garantisce la copertura di tutti gli host [15]. È il caso in cui la correlazione fra segnali eterogenei, provenienti dal codice, dall'infrastruttura e dal contratto, non è un raffinamento ma una condizione necessaria per formulare un giudizio, ed è il criterio che la parte sperimentale di questo lavoro adotta per la categoria.

2.2.5 API5:2023 – Broken Function Level Authorization

Definizione e collocazione Un'API espone operazioni, e non tutte sono destinate a tutti gli utenti. Creare un invito con privilegi amministrativi, esportare l'elenco completo degli utenti, cancellare un account altrui: sono funzioni che l'applicazione riserva a un ruolo, e il controllo che verifica il ruolo del chiamante prima di eseguirle è l'autorizzazione a livello di funzione. API5:2023 raccoglie i casi in cui tale controllo manca o è aggirabile, e in cui un utente anonimo o ordinario riesce di conseguenza a invocare funzioni riservate [24].

La differenza rispetto ad API1:2023 è di livello, ed è l'unica che OWASP fissi in modo prescrittivo. Nella pagina dedicata alla BOLA si legge che l'accesso dell'utente all'endpoint vulnerabile è previsto dal progetto e che la violazione avviene manipolando l'identificatore dell'oggetto, mentre se l'attaccante raggiunge un endpoint o una funzione cui non dovrebbe accedere il caso ricade sotto la BFLA [25]. La domanda da porsi davanti a un finding non riguarda quindi i dati esposti, ma la legittimità della chiamata in sé: chi sostituisce un identificatore su un endpoint cui ha legittimamente accesso e legge il profilo altrui commette una BOLA; chi invoca l'esportazione completa degli utenti commette una BFLA, anche se il risultato include il proprio profilo. La letteratura sul rilevamento nomina le due forme come escalation orizzontale e verticale, intendendo per verticale il caso in cui un utente ordinario assume privilegi amministrativi e per orizzontale quello in cui assume i privilegi di un altro utente ordinario [26]: API5 è la forma verticale.

Il riferimento tassonomico indicato da OWASP è CWE-285 *Improper Authoriza-*

tion [24], che descrive il prodotto che non esegue, o esegue scorrettamente, un controllo di autorizzazione quando un attore tenta di accedere a una risorsa o compiere un'azione [27]. MITRE ne sconsiglia l'uso per mappare vulnerabilità reali, trattandosi di una classe di livello elevato, e rimanda alle figlie: *CWE-862 Missing Authorization* quando il controllo è assente, *CWE-863 Incorrect Authorization* quando esiste ma decide male [27]. La stessa voce dedica una nota terminologica alla distinzione fra autorizzazione e autenticazione, raccomandando le abbreviazioni AuthZ e AuthN separate ed evitando la forma ambigua *Auth* [27]. La distinzione vale anche qui: un'API può autenticare in modo impeccabile e restare priva di autorizzazione funzionale, e un token contraffatto fornisce un'identità, ma è il controllo funzionale mancante a trasformarla in privilegio.

Oltre ad API1, la categoria tocca altre quattro voci dell'elenco, riassunte nella Tabella 2.2.

Categoria	Punto di contatto con API5	Criterio di distinzione
API1	Difetti di autorizzazione su un chiamante autenticato	L'accesso all'endpoint è previsto (API1) oppure indebito (API5); demarcazione fissata da OWASP
API2	Precede API5 nella catena: senza identità non c'è ruolo da verificare	Accertamento dell'identità (AuthN) contro verifica dei privilegi (AuthZ)
API3	Il campo che codifica il ruolo nel corpo della richiesta appartiene a entrambe	Scrivibilità della proprietà (API3) contro raggiungibilità dell'operazione (API5)
API8	Verbi HTTP non necessari e discrepanze fra i server della catena aggirano il controllo	Difetto di configurazione dello stack anziché di logica applicativa
API9	Versioni vecchie ed endpoint deprecati espongono funzioni amministrative protette altrove	Difetto di inventario: il controllo esiste ma non copre tutta la superficie

Tabella 2.2: Relazioni fra API5:2023 e le altre categorie della OWASP API Security Top 10 2023.

Cause e meccanismi tecnici tipici Il controllo funzionale manca perché non ha un luogo obbligato. OWASP osserva che l'autorizzazione su una funzione è gestita tramite configurazione oppure a livello di codice, e che scrivere controlli corretti è un compito confuso, perché le applicazioni moderne contengono molti ruoli, gruppi e gerarchie complesse, con sotto-utenti e utenti titolari di più ruoli [24]. Il singolo confronto è banale: difficile è sapere quale scrivere e ricordarsene ovunque.

Vi si somma il valore predefinito. OWASP prescrive un meccanismo che neghi ogni accesso salvo concessioni esplicite ai singoli ruoli [24], e la prescrizione descrive in negativo ciò che accade di norma: una rotta aggiunta a un controller eredita il routing e l'autenticazione, non l'autorizzazione. Nasce raggiungibile, e tale resta finché qualcuno non decide il contrario.

Sfruttarla non richiede manipolazioni. Bastano chiamate legittime verso un endpoint cui non si dovrebbe accedere, e nelle API il tentativo è più facile perché la struttura è prevedibile: si sostituisce `GET` con `PUT`, oppure `users` con `admins` [24]. La stessa regolarità che rende un'API documentabile la rende enumerabile.

Il primo scenario del documento lo mostra bene. In un'applicazione ad accesso su invito l'app mobile chiama `GET /api/invites/{invite_guid}` e riceve un JSON con ruolo e indirizzo di posta. L'attaccante ne cambia metodo ed endpoint in `POST /api/invites/new`, riservato agli amministratori attraverso una console che non implementa alcun controllo di funzione, e si invia un invito con ruolo `admin`: `201 Created` [24]. Il controllo, in un certo senso, esisteva: stava nella console, cioè nel fatto che solo un amministratore vedesse quel pulsante. L'applicazione ha scambiato la raggiungibilità dell'interfaccia per un diritto, e lo scambio regge finché il client ufficiale è l'unico modo di parlare con l'API. La lettura iniziale, per giunta, aveva già rivelato il campo `role`, cioè la forma della richiesta da costruire. Il secondo scenario è più spiccio: `GET /api/admin/v1/users/all` restituisce i dettagli di tutti gli utenti, e vi arriva chi, appresa la struttura dell'API, tira a indovinare [24].

Restano tre osservazioni che spostano il difetto fuori dal singolo gestore. Il percorso non dice se un endpoint sia amministrativo, perché le funzioni riservate si trovano spesso insieme a quelle ordinarie sotto `api/users` anziché sotto `api/admins` [24]. I verbi HTTP lasciati abilitati per inerzia riaprono l'operazione da una via non presidiata [28], ed è la mossa del primo scenario. Le versioni obsolete, infine, continuano

a servire il gestore precedente al controllo [15].

Impatto essenziale OWASP colloca l’impatto tecnico nella fascia grave: l’attaccante accede a funzionalità non autorizzate, le funzioni amministrative sono il bersaglio privilegiato, e l’esito va dalla divulgazione alla perdita o corruzione dei dati, fino all’interruzione del servizio [24].

Il danno non scala però con il numero di identificatori: scala con la potenza della funzione raggiunta. Una BOLA restituisce un record per richiesta, e arrivare all’intera base dati richiede un’enumerazione che nel traffico si vede; una BFLA su un endpoint di esportazione ottiene lo stesso risultato con una chiamata sola, indistinguibile per volume da qualunque altra. Più la funzione è privilegiata, più l’attacco è silenzioso. E poiché le funzioni riservate sono in prevalenza funzioni di scrittura, lo stesso difetto che espone l’esportazione degli utenti espone, sull’endpoint accanto, la loro cancellazione: l’interruzione di servizio non nasce dall’esaurimento di una risorsa, come in API4:2023, ma da una funzione distruttiva che funziona benissimo, invocata da chi non doveva poterla invocare.

Conta infine la durata. Il primo scenario non finisce con un dato letto, ma con un account amministrativo creato attraverso il flusso legittimo dell’applicazione [24]. Un accesso occasionale diventa un’identità stabile, e le azioni successive sono autenticate, autorizzate e indistinguibili nei registri da quelle di un vero amministratore. Correggere il difetto non revoca l’account: la bonifica si sposta dal codice all’inventario delle utenze.

Mitigazione La prescrizione centrale riguarda il valore predefinito: negare ogni accesso e richiedere concessioni esplicite ai ruoli per ogni funzione [24]. Il vantaggio sta in come si manifesta l’errore. Se il default è il diniego, una dimenticanza produce un 403 e il primo test funzionale la rivela; se il default è il permesso, la stessa dimenticanza non produce alcun segnale.

Segue la revisione degli endpoint, condotta tenendo presenti la logica di business e la gerarchia dei gruppi [24]: nessuna scorciatoia rispetto alla conoscenza del dominio, coerentemente col fatto che il percorso non basta a riconoscere una funzione riservata.

La terza raccomandazione è l'unica con una forma sintattica: i controller amministrativi devono ereditare da un controller astratto che applichi il controllo in base a ruolo o gruppo, e le funzioni amministrative ospitate in un controller ordinario devono applicarlo in proprio [24]. La misura non aggiunge verifiche, le riposiziona. Il controllo diventa una proprietà della gerarchia delle classi, e ometterlo richiede l'atto deliberato di collocare un gestore fuori da quella gerarchia: una discontinuità visibile in revisione e cercabile da uno strumento automatico.

Nessuna delle tre misure risolve la copertura. Il controllo protegge solo la superficie servita dal codice che lo contiene, e restano scoperte le versioni deprecate [15] e le vie alternative lasciate attive [28].

Rilevabilità OWASP giudica facile la rilevabilità: le API sono strutturate e il modo di raggiungere una funzione è prevedibile [24]. Il giudizio riguarda però metà del problema. Formulare la chiamata candidata è facile; stabilire se la risposta fosse dovuta non lo è, perché un 200 OK attesta che l'operazione è stata eseguita, non che fosse legittimo eseguirla. Nemmeno il percorso aiuta, come avverte la fonte primaria [24]: una regola che segnali i soli gestori sotto `api/admins` privi di verifica manca per costruzione i casi più comuni.

Il contributo della letteratura non è ispezionare il codice ma ricostruire la specifica mancante. Sun, Xu e Su la ricavano dalla navigazione, derivando dalle pagine raggiungibili da ciascun ruolo le regole implicite dell'applicazione [29]. RoleCast la ricava dalla forma del codice: poiché dentro una partizione per ruolo i controlli sono applicati in modo coerente, da quella coerenza il sistema inferisce quali condizioni siano controlli di sicurezza e segnala i percorsi in cui mancano [30]. MACE ragiona sul contesto di autorizzazione e copre entrambe le direzioni dell'escalation su applicazioni di dimensioni reali [26]. Atlidakis, Godefroid e Polishchuk rinunciano a inferire e scrivono: definiscono regole di sicurezza per le API REST e le verificano con fuzzing stateful guidato dalla descrizione dell'interfaccia [31]. Il lavoro più recente affronta lo stesso problema in black box [32].

2.2.6 API6:2023 – Unrestricted Access to Sensitive Business Flows

Definizione e collocazione Le categorie esaminate finora descrivono controlli assenti. Qui ogni singola chiamata è legittima: l'utente ha diritto di compierla, i controlli di autorizzazione funzionano, il codice si comporta come progettato. Il danno nasce dal numero. OWASP definisce vulnerabile l'endpoint che espone un flusso di business sensibile senza limitarne adeguatamente l'accesso, portando come esempi l'acquisto di un prodotto, la creazione di un commento e la prenotazione di una fascia oraria [13].

Segue l'affermazione che definisce davvero la categoria: il rischio di un accesso eccessivo cambia da settore a settore, e la creazione di post tramite script può costituire spam per un social network ed essere incoraggiata da un altro [13]. Nessuna proprietà del codice, del contratto o della configurazione rende dunque un endpoint vulnerabile ad API6: lo stesso identico endpoint è sicuro in un contesto e vulnerabile in un altro, e a decidere è il modello di business che gli sta dietro.

Cause e meccanismi tecnici tipici La causa che OWASP indica non è tecnica: manca una visione d'insieme dell'API che ne supporti i requisiti di business [13]. Il difetto nasce nel raccordo fra chi progetta l'endpoint e chi conosce il valore economico di ciò che l'endpoint muove, e dove quel raccordo manca nessuno è nella posizione di accorgersene.

Lo sfruttamento richiede di comprendere il modello di business, individuare i flussi sensibili e automatizzarne l'accesso [13]: soltanto l'ultimo passo produce traffico osservabile. Il primo scenario lo illustra con una console da gioco annunciata in quantità limitata, acquistata da un programma eseguito il giorno del lancio e distribuita su indirizzi IP e località diverse [13]. La distribuzione geografica non è un dettaglio di colore: è la contromisura dell'attaccante contro il limite per indirizzo, e spiega perché il rate limiting nella sua forma più semplice non basti.

Impatto essenziale Su questo punto il documento è internamente incoerente. La riga dei rating classifica l'impatto tecnico come moderato, mentre la descrizione degli impatti afferma che un impatto tecnico in generale non è atteso e porta come esempi di danno l'impossibilità per gli utenti legittimi di acquistare un prodotto e

l'inflazione nell'economia interna di un videogioco [13]. È la seconda formulazione a corrispondere agli scenari descritti, che non producono né violazione di riservatezza né alterazione di dati né indisponibilità del servizio. Ciò che cambia sta fuori dal sistema e si misura in scorte sottratte, ricavi persi o costi di acquisizione pagati per utenti inesistenti. Ne segue un effetto collaterale rilevante: un incidente che non produce indisponibilità né perdita di dati non attiva le procedure di risposta e viene percepito come problema commerciale anziché di sicurezza.

Mitigazione OWASP propone una struttura a due livelli in cui l'ordine conta: prima si identificano, sul piano del business, i flussi che un uso eccessivo può danneggiare, e solo dopo si sceglie il meccanismo tecnico adeguato al rischio individuato [13]. Nessuna categoria precedente prescrive una fase preliminare simile, perché in nessuna altra la configurazione corretta dipende dall'azienda che la adotta. I meccanismi elencati, dal fingerprinting del dispositivo al captcha, dall'individuazione di comportamenti non umani come il passaggio dal carrello all'acquisto in meno di un secondo fino al blocco dei nodi di uscita Tor, sono presentati come modi per rallentare le minacce automatizzate, non per impedirle [13]. La logica che li accomuna è economica: nessuno produce una decisione binaria affidabile, tutti spostano il costo dell'attacco fino a superare il profitto atteso.

Rilevabilità — stato dell'arte in letteratura Il rilevamento automatico si scontra qui con l'assenza di un oracolo: nessun artefatto analizzabile, che sia codice, contratto o configurazione, risponde alla domanda se un dato flusso in una data azienda vada protetto, e la mancanza di una mappatura CWE registra il fatto che non esiste una debolezza del software da nominare. Le tecniche note per i difetti di logica applicativa cercano deviazioni da un comportamento atteso [33, 34], mentre qui il flusso viene percorso correttamente e l'attacco sta nel ripeterlo. Restano le contromisure comportamentali a runtime indicate da OWASP [13], che riconoscono l'automazione durante l'attacco anziché la vulnerabilità prima che avvenga.

2.2.7 API7:2023 – Server Side Request Forgery

Definizione e collocazione Nelle categorie viste finora l'attaccante agisce sui dati che l'API restituisce. In API7:2023 agisce sulle richieste che l'API emette. Il difetto compare quando un'API recupera una risorsa remota senza validare l'URL fornito dall'utente: l'attaccante può allora costringere l'applicazione a inviare una richiesta verso una destinazione inattesa, anche se protetta da un firewall o da una VPN [35]. Il server diventa un intermediario involontario, che agisce per conto di chi lo interroga ma con i propri privilegi di rete. Non viola quindi un controllo di accesso applicativo: ne annulla uno di rete.

MITRE classifica CWE-918 come debolezza *resultant*, legata cioè alla presenza di altre [36]: l'SSRF è raramente il fine dell'attacco, quasi sempre il mezzo. A renderla attuale è il contesto in cui le API operano. È più comune, perché webhook, recupero di file da URL e anteprime dei collegamenti spingono a raggiungere risorse esterne indicate dall'utente; ed è più pericolosa, perché fornitori cloud, Kubernetes e Docker espongono canali di gestione su HTTP a percorsi noti, mentre limitare il traffico in uscita resta difficile [35]. I rapporti con le categorie contigue, fra cui la contromisura condivisa con API10:2023 [14], sono riassunti nella Tabella 2.3.

Categoria	Punto di contatto con API7	Criterio di distinzione
API3	L'URL non validato è una proprietà dell'oggetto in ingresso	Esito: alterazione di un dato (API3) contro alterazione del comportamento di rete (API7)
API4	Timeout sulle chiamate in uscita; l'SSRF può causare denial of service	Difetto primario: assenza di limiti (API4) contro destinazione scelta dall'attaccante (API7)
API10	Entrambe riguardano il traffico in uscita; comune il divieto di seguire redirezioni	Chi sceglie la destinazione: l'attaccante (API7) contro un fornitore legittimo ma non validato (API10)

Tabella 2.3: Relazioni fra API7:2023 e le categorie contigue della OWASP API Security Top 10 2023.

Cause e meccanismi tecnici tipici La causa immediata è l'assenza o l'inadeguatezza della validazione dell'URI fornito dal client, e sono i modelli di sviluppo moderni a incoraggiare l'accesso a URI di provenienza esterna [35]. Il difetto nasce quindi da una funzionalità voluta: nessuno introduce per sbaglio un endpoint che scarica un'immagine da un indirizzo scelto dall'utente.

I due scenari del documento mostrano le due forme dell'attacco. Nella forma cieca un social network consente di indicare l'immagine del profilo tramite URL; l'attaccante scrive `localhost:8080` nel campo `picture_url` e, misurando il tempo di risposta, deduce se la porta sia aperta, scandendo così la rete interna [35]. L'informazione non arriva dal corpo della risposta, ma dalla sua latenza. Nella forma con ritorno un prodotto di sicurezza invia una richiesta di prova al webhook che l'utente sta configurando e gliene mostra la risposta; l'attaccante vi indica il servizio di metadati dell'ambiente cloud e ne ottiene le credenziali [35]. Il webhook rende comune il difetto, l'indirizzo di metadati a percorso noto lo rende grave.

Una terza famiglia riguarda non l'assenza della validazione ma il suo aggiramento — formati alternativi di un indirizzo IP, redirezioni verso host interni, schemi come `file://` [36]. Validare un URL è insomma un problema di parsing, che le espressioni regolari scritte a mano risolvono male. Un'analisi su oltre 27.000 applicazioni PHP mostra però che la causa più diffusa non è una difesa mal costruita, ma l'assenza di qualunque difesa [37].

Impatto essenziale OWASP elenca come esiti l'enumerazione dei servizi interni per scansione di porte, la divulgazione di informazioni, l'aggiramento di firewall e, in alcuni casi, il denial of service o l'uso del server come proxy [35].

L'elemento che distingue l'SSRF è il cambio di perimetro: l'attaccante non ottiene un dato, ottiene una posizione, quella del server. Il valore dell'attacco dipende perciò da cosa sia raggiungibile dall'interno: in un'architettura piatta un solo endpoint vulnerabile annulla la protezione di tutti i servizi che si fidano del perimetro, in una segmentata consente poco più di una scansione.

I servizi di metadati cloud convertono poi l'SSRF da ricognizione a compromissione: rispondono a un indirizzo fisso e noto, non richiedono autenticazione perché

presuppongono che solo l'istanza possa raggiungerli, ed espongono le credenziali del ruolo associato alla macchina, e MITRE ne registra casi sfruttati attivamente [36].

Mitigazione Sul piano della rete OWASP raccomanda di isolare il meccanismo di recupero delle risorse, che di norma mira a risorse remote e non interne [35]. È la difesa più robusta perché non dipende dalla correttezza della validazione: se il componente che effettua le richieste in uscita non ha rotta verso la rete interna, un URL malevolo non produce nulla.

Sul piano applicativo il documento prescrive liste di elementi ammessi su tre dimensioni — origini remote attese, schemi di URL e porte, tipi di media accettati [35] — ciascuna delle quali blocca una famiglia di aggiramenti diversa. Seguono la disabilitazione delle redirezioni, un parser di URL collaudato e il divieto di restituire al client le risposte grezze [35]: quest'ultima misura non impedisce l'attacco, ma ne degrada l'esito dalla forma con ritorno a quella cieca.

Va infine registrata un'ammissione che nessun'altra categoria contiene: il rischio SSRF non può sempre essere eliminato del tutto [35], perché funzionalità e vulnerabilità coincidono. Un servizio che offre anteprime di collegamenti arbitrari deve per definizione contattare indirizzi scelti dall'utente: se ne può confinare l'effetto, non impedirli.

Rilevabilità: stato dell'arte in letteratura Per la prima volta nel corso del capitolo le due fonti primarie concordano, e a favore dell'analisi statica. OWASP giudica facile la rilevabilità, precisando che la forma cieca richiede maggiore sforzo [35]; CWE-918 indica come metodo la sola analisi statica automatica, intesa come ricerca dei percorsi che connettono una sorgente di input a un sink, e ne dichiara alta l'efficacia [36].

La differenza rispetto alle categorie precedenti sta in dove si trovi l'oracolo. Per API5 lo strumento vede l'assenza di un controllo ma non sa se fosse dovuto; per API4 vede l'assenza di un limite ma non sa se sia applicato altrove. Per API7 la domanda non è se qualcosa manchi, ma se due punti del programma siano connessi: una proprietà verificabile senza conoscere le intenzioni del progettista.

La letteratura conferma il quadro e ne segnala i limiti: Wessels e colleghi validano a mano i risultati di un'analisi di flusso sul codice PHP [37], Artemis pota i

percorsi non sfruttabili per ridurre i falsi positivi [38], SSRFuzz affianca il fuzzing alla marcatura dinamica [39]. Se ne ricavano i tre problemi che ogni rilevatore statico deve affrontare, validi anche per il modulo descritto nella parte sperimentale: l’inventario dei sink, perché ometterne uno produce un falso negativo silenzioso; la sovra-marcatura, perché non ogni stringa contaminata che raggiunge un sink produce un URL controllabile; e la validazione intermedia, perché distinguere un controllo efficace da uno aggirabile richiede di ragionare sulla sua semantica.

La distinzione fra le due forme ricade infine sulla verifica. Con ritorno la conferma dinamica è immediata: si punta l’endpoint verso un indirizzo controllato e si osserva se la richiesta arriva; per la forma cieca serve un canale laterale, tipicamente una risoluzione DNS. Nella pipeline adottata nel seguito l’assenza di conferma dinamica non deve quindi declassare il finding statico: significa soltanto che il canale di ritorno non è stato osservato.

2.2.8 API8:2023 – Security Misconfiguration

Definizione e collocazione Le categorie viste finora individuano un difetto in un punto preciso: un controllo che manca in un handler, un limite non impostato, un URL non validato. API8:2023 non ha un punto. OWASP dichiara che la configurazione insicura può verificarsi a qualunque livello dello stack dell’API, dal livello di rete a quello applicativo [28], e l’elenco degli indicatori di vulnerabilità attraversa effettivamente tutti quei livelli: hardening assente in una qualsiasi parte dello stack o permessi mal configurati sui servizi cloud, patch di sicurezza mancanti o sistemi non aggiornati, funzionalità non necessarie abilitate come verbi HTTP o funzioni di logging, discrepanze nel modo in cui i server della catena HTTP elaborano le richieste in arrivo, TLS assente, direttive di sicurezza o di controllo della cache non inviate ai client, politica CORS mancante o impostata male, messaggi di errore che includono tracce dello stack o espongono altre informazioni sensibili [28].

Sono otto voci che hanno in comune soltanto una cosa: nessuna di esse è un errore di logica applicativa. In ciascun caso il codice fa quello che deve; è l’ambiente in cui gira, o un’impostazione che qualcuno ha lasciato al valore predefinito, a produrre l’esposizione. Questa è la definizione operativa della categoria, e ne spiega tanto

l'ampiezza quanto la difficoltà di trattarla come le altre.

La particolarità tassonomica lo conferma nel modo più netto. Alla categoria OWA-SP associa sette voci CWE, un numero superiore a quello di qualunque altra voce della Top 10 [28]. Il problema è che diverse fra esse non sono debolezze. CWE-16 *Configuration* è una categoria, e MITRE ne dichiara la mappatura *prohibited*, ossia proibita, motivandola con il fatto che si tratta di un raggruppamento informale e non di una debolezza in sé, e aggiungendo che l'uso di categorie per la mappatura è sconsigliato dal 2019 [40]. La nota di manutenzione della stessa voce è quanto di più esplicito si possa desiderare: MITRE riconosce che varie fonti vi mappano comunque nonostante la guida contraria, osserva che in questo caso non esistono debolezze CWE chiare da utilizzare, e nota che una formulazione come *Inappropriate Configuration* suonerebbe più conforme allo stile CWE ma continuerebbe a non indicare un comportamento effettivo del prodotto [40]. L'indicazione operativa che ne segue è di analizzare la debolezza nel comportamento che la configurazione ha determinato, riconducendola per esempio alla discendenza di CWE-284 per il controllo di accesso o a CWE-400 per la gestione delle risorse [40].

Ne discende un criterio che vale la pena fissare, perché ha conseguenze su come un difetto di configurazione va classificato. La configurazione non è una debolezza: è il meccanismo attraverso cui una debolezza viene introdotta. Un'impostazione CORS permissiva non è insicura in astratto, lo diventa perché produce un difetto di controllo di accesso; l'assenza di TLS non è una debolezza autonoma, ma la causa di una trasmissione in chiaro di informazioni sensibili, che è CWE-319 ed è una debolezza vera. Delle sette voci elencate da OWASP, quelle utilizzabili per una mappatura sono infatti quelle che nominano un comportamento e non un ambito: CWE-209 sulla generazione di messaggi d'errore contenenti informazioni sensibili, CWE-319 sulla trasmissione in chiaro, CWE-444 sull'interpretazione incoerente delle richieste HTTP, CWE-942 sulla politica cross-domain permissiva verso domini non fidati [28].

Le metriche assegnate collocano API8 fra le categorie più gravi dell'elenco: sfruttabilità facile, prevalenza diffusa, rilevabilità facile, impatto tecnico severo [28]. È una combinazione che nella Top 10 condivide con poche altre voci, e la motivazione data per il vettore d'attacco chiarisce perché: gli attaccanti cercano difetti non corretti,

endpoint comuni, servizi in esecuzione con configurazioni predefinite insicure, file e directory non protetti, e OWASP osserva che si tratta in larga parte di conoscenza pubblica per la quale possono essere disponibili exploit [28]. Non serve comprendere l'applicazione: serve sapere cosa cercare, e quell'informazione è documentata.

Il rapporto con le altre categorie è, per API8, di tipo particolare: non di parentela ma di sovrapposizione parziale ricorrente. Il difetto di configurazione compare come causa alternativa in almeno tre voci già trattate.

Con API4:2023 il contatto è sui limiti dichiarativi. I vincoli su memoria, CPU e processi che OWASP raccomanda di affidare a container e funzioni serverless [12] vivono in file di orchestrazione, e la loro assenza è tanto un difetto di consumo di risorse quanto una configurazione carente nel senso di API8, che chiede espressamente di rivedere i file di orchestrazione e i servizi cloud [28].

Con API5:2023 il contatto è sul metodo HTTP, già osservato in quella sede. La raccomandazione di API8 di specificare quali verbi siano ammessi per ciascuna API e di disabilitare tutti gli altri [28] previene per via configurativa il caso in cui una rotta risulti protetta in lettura e scoperta in cancellazione. La differenza sta nel livello a cui il rimedio agisce: nell'applicazione per API5, nella catena dei server per API8.

Con API7:2023 il contatto riguarda la politica di uscita. Lo scenario che OWASP porta per API8 lo mostra con chiarezza, poiché l'esecuzione di codice remoto vi diventa possibile per la combinazione di una configurazione predefinita insicura dell'utilità di logging e di una politica di rete in uscita permissiva [28]: la seconda condizione è esattamente quella che rende sfruttabile anche l'SSRF. La Tabella 2.4 riassume.

Categoria	Punto di contatto con API8	Criterio di distinzione
API4	Limiti di risorse dichiarati nei file di orchestrazione	Difetto di consumo (API4) contro impostazione mancante nello stack (API8)
API5	Verbi HTTP non necessari abilitati sulla catena dei server	Livello del rimedio: applicazione (API5) contro configurazione dei server (API8)
API7	Politica di rete in uscita permissiva come preconditione dello sfruttamento	Origine della richiesta: input non validato (API7) contro assenza di segmentazione (API8)

Tabella 2.4: Sovrapposizioni fra API8:2023 e le categorie contigue della OWASP API Security Top 10 2023.

Cause e meccanismi tecnici tipici La causa strutturale è che la configurazione di un'API moderna non risiede in un luogo unico né sotto una responsabilità unica. Si distribuisce fra il codice applicativo, i file di orchestrazione, il reverse proxy, il gateway, le impostazioni del fornitore cloud e i valori predefiniti delle librerie di terze parti, e ciascuno di questi livelli ha un proprietario diverso e un ciclo di vita proprio. Un'impostazione corretta al momento del rilascio può cessare di esserlo dopo un aggiornamento che nessuno ha collegato alla sicurezza.

Il primo scenario di OWASP illustra il meccanismo dei valori predefiniti pericolosi, e lo fa con quello che è riconoscibilmente il caso Log4Shell. Un server di back-end tiene un registro degli accessi scritto da una diffusa utilità di logging open source che supporta l'espansione di segnaposto e le ricerche JNDI, entrambe abilitate per impostazione predefinita; per ogni richiesta viene scritta una riga secondo un formato che include il percorso e il codice di stato. L'attaccante invia una richiesta apparentemente innocua verso `/health` con un'intestazione `X-Api-Version` contenente un riferimento JNDI a un server LDAP che controlla, e nel momento in cui l'utilità espande quel valore per scriverlo nel registro scarica ed esegue l'oggetto malevolo dal server remoto [28]. Vale la pena osservare cosa manca in questa catena: non c'è alcun difetto nel codice dell'applicazione, non c'è validazione dell'input che avrebbe

potuto essere presente, e l'endpoint colpito è quello di verifica dello stato di salute, tipicamente il più innocuo dell'intera API.

Il secondo scenario mostra il meccanismo opposto, quello dell'impostazione assente. Un social network espone i messaggi privati tramite una chiamata a `/dm/user_updates.json` con identificativo di conversazione e cursore; poiché la risposta non include l'intestazione `Cache-Control`, le conversazioni private finiscono nella cache del browser, da cui possono essere recuperate leggendo i file sul filesystem [28]. Qui non c'è nulla di attivo da disabilitare: c'è una direttiva che nessuno ha inviato, e la conseguenza è che dati riservati vengono conservati in chiaro su una macchina che l'applicazione non controlla.

Un terzo meccanismo, che gli scenari non illustrano ma che l'elenco degli indicatori nomina, riguarda le discrepanze nell'elaborazione delle richieste lungo la catena dei server [28]. È il fenomeno che la voce CWE-444 chiama interpretazione incoerente delle richieste HTTP e che la pratica conosce come *request smuggling*: quando un bilanciatore di carico e un server applicativo delimitano diversamente il confine fra due richieste consecutive sulla stessa connessione, un attaccante può far sì che parte del proprio traffico venga attribuita alla richiesta di un altro utente. Il difetto non risiede in nessuno dei due componenti presi singolarmente, ma nella loro combinazione, ed è la ragione per cui OWASP raccomanda di assicurarsi che tutti i server della catena elaborino le richieste in maniera uniforme [28].

Resta la causa più banale e più diffusa, quella che l'elenco degli indicatori mette al secondo posto: patch mancanti e sistemi non aggiornati [28]. Non è un errore di configurazione in senso stretto ma di manutenzione, e la sua presenza nella stessa categoria segnala che il confine fra le due cose, nella pratica operativa, è poco netto.

Impatto essenziale La formulazione di OWASP è breve e va letta nei suoi due termini: le configurazioni insicure non solo espongono dati sensibili degli utenti, ma anche dettagli del sistema che possono condurre alla compromissione completa del server [28]. La seconda parte è quella che giustifica la classificazione dell'impatto tecnico come severo, e i due scenari ne mostrano i due estremi: nel secondo il danno è l'esposizione di conversazioni private, nel primo è l'esecuzione di codice arbitrario sul server.

Ciò che distingue l'impatto di API8 è la sua natura abilitante. Un difetto di configurazione raramente costituisce l'attacco: costituisce la condizione che rende praticabile un altro attacco, o che ne amplifica l'esito. Una traccia di stack in un messaggio d'errore non compromette nulla di per sé, ma rivela versioni, percorsi interni e talvolta frammenti di query, riducendo il costo della ricognizione; una politica di uscita permissiva non è sfruttabile direttamente, ma trasforma un SSRF cieco in un SSRF con ritorno. Questo spiega perché la categoria compaia come preconditione in tante altre voci dell'elenco e perché il suo impatto sia difficile da valutare in isolamento.

Un secondo tratto riguarda l'ampiezza. Poiché la configurazione è per sua natura trasversale, un singolo difetto non riguarda un endpoint ma tutti gli endpoint serviti da quel componente. L'intestazione di cache mancante nel secondo scenario non affligge una conversazione: affligge ogni risposta che attraversa quella configurazione.

Mitigazione La struttura che OWASP propone distingue due piani. Sul piano del processo, il ciclo di vita dell'API dovrebbe comprendere un procedimento di hardening ripetibile che consenta un rilascio rapido e agevole di un ambiente adeguatamente irrigidito, un'attività di revisione e aggiornamento delle configurazioni sull'intero stack che includa i file di orchestrazione, i componenti dell'API e i servizi cloud, e un processo automatizzato che valuti in modo continuo l'efficacia di configurazioni e impostazioni in tutti gli ambienti [28]. I tre requisiti hanno un'implicazione comune che merita di essere resa esplicita: la configurazione va trattata come artefatto versionato e verificabile, non come stato accumulato di una macchina.

Sul piano dei controlli specifici, il documento elenca sei misure [28]. La comunicazione fra client e server e verso ogni componente a monte o a valle deve avvenire su canale cifrato, indipendentemente dal fatto che l'API sia interna o esposta al pubblico, precisazione che chiude l'assunto per cui il traffico interno sarebbe fidato. I verbi HTTP ammessi vanno specificati per ciascuna API e tutti gli altri disabilitati. Le API destinate a client basati su browser devono implementare una politica CORS adeguata e includere le intestazioni di sicurezza applicabili. I tipi di contenuto e i formati accettati in ingresso vanno ristretti a quelli che soddisfano i requisiti funzionali.

Tutti i server della catena devono elaborare le richieste in modo uniforme per evitare problemi di desincronizzazione. Infine, dove applicabile, vanno definiti e applicati gli schemi di tutti i payload di risposta, comprese le risposte di errore, per impedire che tracce di eccezione e altre informazioni di valore raggiungano l'attaccante.

Quest'ultima misura merita una nota perché collega la mitigazione al contratto. Uno schema di risposta dichiarato per le condizioni di errore non è documentazione: è un vincolo che, se applicato a runtime, impedisce strutturalmente la fuga di informazioni diagnostiche. È il caso in cui la specifica dell'API funge da controllo di sicurezza e non da suo commento.

Rilevabilità: stato dell'arte in letteratura OWASP classifica la rilevabilità come facile e ne dà una motivazione che, per la prima volta in questo capitolo, rinvia direttamente agli strumenti: sono disponibili strumenti automatici per individuare e sfruttare configurazioni errate come servizi non necessari o opzioni obsolete [28]. La formulazione è simmetrica e va notata: gli stessi strumenti servono a chi difende e a chi attacca, perché ciò che cercano è pubblicamente noto.

La ragione per cui il rilevamento funziona bene si comprende per contrasto con le categorie precedenti. Per l'autorizzazione funzionale lo strumento non sapeva se un controllo fosse dovuto; per il consumo di risorse non sapeva se un limite fosse applicato altrove; per i flussi di business non aveva alcun artefatto da interrogare. Per la configurazione l'oracolo è esterno e pubblico: esistono valori raccomandati, elenchi di impostazioni predefinite pericolose, versioni note come vulnerabili, e verificare la conformità di una configurazione rispetto a un riferimento è un problema di confronto, non di inferenza. È il motivo per cui questa è l'unica categoria della Top 10 per cui esiste un mercato maturo di strumenti che operano su artefatti dichiarativi anziché su codice.

Il quadro presenta però tre limiti che vanno riconosciuti.

Il primo riguarda la copertura degli artefatti. La configurazione risiede in file di orchestrazione, manifest di deployment, definizioni infrastrutturali, impostazioni del gateway, valori predefiniti delle librerie e stato effettivo delle risorse cloud; nessuno strumento li legge tutti, e ciò che non è dichiarato in un file non è ispezionabile prima dell'esecuzione. OWASP stessa, chiedendo un processo automatizzato che valuti

l'efficacia delle impostazioni in tutti gli ambienti [28], riconosce implicitamente che la verifica sugli artefatti non basta e va accompagnata dall'osservazione dell'ambiente reale.

Il secondo riguarda la deriva. Un'analisi statica di un file infrastrutturale attesta ciò che quel file dichiara, non ciò che è effettivamente attivo sul sistema; ogni modifica applicata fuori dalla pipeline, e ogni impostazione modificata manualmente in produzione, apre una divergenza che l'analisi non vede. Il difetto non è dello strumento ma della premessa: la configurazione descritta e quella effettiva sono due oggetti distinti.

Il terzo riguarda il contesto. Una politica CORS che ammette qualunque origine è un difetto per un'API destinata a client autenticati e una scelta legittima per un'API pubblica di sola lettura; un'intestazione di cache assente è irrilevante su una risposta pubblica e grave su una privata, come mostra il secondo scenario OWASP. La verifica sintattica di un'impostazione, in altri termini, non stabilisce se essa sia sbagliata: stabilisce se si discosta da un riferimento, e la pertinenza di quel riferimento dipende dalla funzione dell'endpoint. È una forma attenuata della difficoltà già incontrata per i flussi di business, con la differenza che qui il contesto necessario è tecnico e ricavabile in parte dal contratto, non economico e del tutto esterno agli artefatti.

Resta un'osservazione sulla natura del segnale. Per le categorie di autorizzazione l'analisi cercava l'assenza di qualcosa che avrebbe dovuto esserci; per API8 cerca la presenza di qualcosa che non avrebbe dovuto esserci, come un verbo abilitato o una direttiva permissiva, oppure l'assenza di qualcosa che ha un valore corretto noto, come un'intestazione di sicurezza. Sono due problemi asimmetrici: il secondo si presta a una verifica per confronto con un riferimento e produce falsi positivi contestuali ma pochi falsi negativi, mentre il primo richiede di ricostruire l'intenzione del progettista. Questa asimmetria spiega perché la stessa infrastruttura di analisi statica dia risultati di qualità molto diversa a seconda della categoria a cui viene applicata.

hiarate sono candidate a essere API ombra, cioè funzionalità raggiungibili che nessun documento menziona e che quindi sfuggono a ogni processo di revisione basato sul contratto. Le rotte dichiarate ma non implementate sono, all'opposto, un indizio di documentazione obsoleta, e possono segnalare una dismissione avvenuta

senza aggiornare la specifica oppure una specifica scritta in anticipo e mai realizzata. Nessuna delle due differenze è di per sé una vulnerabilità, ma entrambe misurano la divergenza fra il modello e la realtà, che è esattamente il difetto che la categoria descrive.

Questo confronto ha però un limite intrinseco che va dichiarato: opera fra due rappresentazioni della stessa applicazione, e non dice nulla sugli host su cui quell'applicazione è distribuita. Il primo scenario di OWASP è precisamente il caso in cui codice e contratto coincidono perfettamente e il difetto risiede altrove, nella topologia di rete. Rilevare quel caso richiede di enumerare gli host effettivamente raggiungibili e di confrontarli con l'inventario dichiarato, che è un'attività di ricognizione infrastrutturale e non di analisi del software.

La letteratura accademica su questo problema specifico è scarsa, e vale la pena dirlo esplicitamente anziché colmare il vuoto con riferimenti impropri. La ragione plausibile è che il problema non si presta alla forma sperimentale consueta della ricerca sulla sicurezza del software: non esiste un insieme di verità di riferimento contro cui misurare un rilevatore di API ombra, perché costruire quell'insieme richiederebbe di conoscere l'inventario completo di organizzazioni reali, che è precisamente l'informazione che per ipotesi non esiste. Il campo è quindi presidiato da strumenti commerciali di scoperta delle API, che operano tipicamente osservando il traffico di produzione, e le cui prestazioni sono documentate da materiale dei fornitori e non da valutazioni indipendenti sottoposte a revisione paritaria.

Ne discende una conclusione sulla natura del segnale che chiude la trattazione di questa categoria. Per API9 non esiste un rilevamento nel senso in cui esiste per le categorie precedenti, cioè l'individuazione di un difetto in un artefatto. Esiste la misura di una divergenza fra rappresentazioni indipendenti dello stesso sistema — il contratto, il codice, l'inventario degli host, il traffico osservato — e ogni divergenza è un indizio la cui gravità dipende da cosa si trovi dall'altra parte. Il valore di questa misura non sta nel produrre un verdetto ma nel restituire visibilità: ciò che il difetto sottrae per definizione.

2.2.9 API10:2023 – Unsafe Consumption of APIs

Definizione e collocazione Le nove categorie esaminate finora hanno un tratto comune che diventa visibile solo quando si incontra la decima: descrivono tutte l'API come bersaglio di un attacco diretto. API10:2023 rovescia la prospettiva. Qui l'attaccante non colpisce l'API bersaglio ma un servizio da cui essa dipende, e il danno arriva attraverso un canale che l'applicazione considera fidato. Le note di rilascio dell'edizione 2023 dichiarano di aver introdotto la categoria per affrontare un fenomeno osservato di recente, ossia il fatto che gli attaccanti hanno cominciato a cercare i servizi integrati di un bersaglio per compromettere quelli, anziché colpire direttamente le API del bersaglio stesso [41].

La causa che OWASP individua è di ordine psicologico prima che tecnico, e il documento la enuncia senza mediazioni: gli sviluppatori tendono a fidarsi dei dati ricevuti da API di terze parti più di quanto si fidino dell'input dell'utente, e questo vale in particolare per le API offerte da aziende note; di conseguenza tendono ad adottare standard di sicurezza più deboli, per esempio in materia di validazione e sanificazione dell'input [14]. La stessa osservazione ritorna nella descrizione della debolezza, dove si legge che gli sviluppatori tendono a fidarsi senza verificare degli endpoint che interagiscono con API esterne o di terze parti, affidandosi a requisiti più deboli riguardo alla sicurezza del trasporto, all'autenticazione e autorizzazione, e alla validazione e sanificazione dell'input [14].

Il documento elenca cinque condizioni che rendono l'API vulnerabile: interagire con altre API su un canale non cifrato; non validare e sanificare adeguatamente i dati raccolti da altre API prima di elaborarli o di passarli a componenti a valle; seguire ciecamente le redirezioni; non limitare la quantità di risorse disponibili per elaborare le risposte dei servizi di terze parti; non implementare timeout per le interazioni con essi [14].

Su questo elenco vale la pena registrare un'obiezione sollevata pubblicamente sul repository del progetto durante la fase di revisione, secondo cui almeno tre di quelle voci apparirebbero concettualmente ad altre categorie: il limite alle risorse per elaborare le risposte ricade nel consumo incontrollato di risorse, mentre l'interazione su canale non cifrato ricade nella configurazione insicura [42]. L'osservazione è fondata

e il gruppo di lavoro non l'ha recepita, sicché l'elenco pubblicato mescola condizioni che appartengono al nucleo della categoria, come la mancata validazione dei dati ricevuti e le redirezioni seguite ciecamente, con altre che sono la manifestazione di difetti già trattati altrove nella direzione del traffico in uscita.

I riferimenti esterni indicati sono tre — CWE-20 sulla validazione impropria dell'input, CWE-200 sull'esposizione di informazioni sensibili a un attore non autorizzato, CWE-319 sulla trasmissione in chiaro [14] — e la loro genericità riflette una proprietà reale della categoria anziché una scelta approssimativa. OWASP scrive infatti che l'impatto varia a seconda di ciò che l'API bersaglio fa con i dati che ha prelevato [14]: la debolezza specifica dipende dalla destinazione del dato non validato, che può essere una query SQL, un motore di template o un componente a valle qualsiasi. API10 individua quindi un punto d'ingresso, non un difetto terminale.

Le metriche assegnate presentano una tensione interna che merita di essere segnalata. La sfruttabilità è dichiarata facile, ma la descrizione del vettore afferma che lo sfruttamento richiede all'attaccante di identificare e potenzialmente compromettere altre API o servizi con cui l'API bersaglio è integrata, aggiungendo che di norma tale informazione non è pubblicamente disponibile o il servizio integrato non è facilmente attaccabile [14]. La prevalenza è comune, la rilevabilità media e l'impatto tecnico severo [14]. Se ne ricava che il giudizio di facilità riguarda la fase finale dell'attacco, quella in cui il dato malevolo viene consegnato e accettato senza controlli, mentre la fase preparatoria resta la parte costosa.

Il rapporto con le categorie contigue è definito dalla direzione del traffico e dalla natura della fiducia mal riposta.

Con API7:2023 la simmetria è quella già osservata in quella sede: entrambe riguardano richieste che il server emette, ma in API7 la destinazione è scelta dall'attaccante attraverso un input non validato, mentre in API10 la destinazione è legittima e il problema riguarda ciò che torna indietro. Le due categorie condividono la contromisura del divieto di seguire redirezioni automatiche [35, 14].

Con API4:2023 e con API8:2023 i rapporti sono quelli evidenziati dall'obiezione riportata sopra: i timeout e i limiti sulle risposte appartengono al consumo di risorse, la cifratura del canale alla configurazione. In entrambi i casi la specificità di API10 non sta nel controllo mancante ma nella sua destinazione, cioè nel fatto che il componente

non protetto sia rivolto verso un fornitore anziché verso un client.

Con API9:2023 il legame è complementare e riguarda il medesimo oggetto visto da due lati. Il punto cieco dei flussi di dati descritto in quella categoria concerne ciò che l'organizzazione condivide con terze parti senza saperlo; API10 concerne ciò che riceve indietro senza verificarlo. Non a caso la prima raccomandazione di API9 sul secondo inventario chiede di censire i servizi integrati documentandone ruolo, dati scambiati e sensibilità [15]: quell'inventario è la preconditione di qualunque valutazione della postura di sicurezza dei fornitori, che è a sua volta la prima contromisura prescritta da API10. La Tabella 2.5 riassume.

Categoria	Punto di contatto con API10	Criterio di distinzione
API4	Timeout e limiti sulle risposte dei servizi integrati	Bene protetto: le risorse del server (API4) contro l'integrità del dato ricevuto (API10)
API7	Entrambe riguardano il traffico in uscita e vietano le redirezioni cieche	Destinazione scelta dall'attaccante (API7) contro fornitore legittimo ma non verificato (API10)
API8	Interazione con servizi terzi su canale non cifrato	Impostazione dello stack (API8) contro fiducia mal riposta nella dipendenza (API10)
API9	Inventario dei servizi integrati e dei dati scambiati	Cosa si condivide senza saperlo (API9) contro cosa si accetta senza verificarlo (API10)

Tabella 2.5: Relazioni fra API10:2023 e le categorie contigue della OWASP API Security Top 10 2023.

Cause e meccanismi tecnici tipici La causa strutturale è un confine di fiducia collocato nel punto sbagliato. L'architettura difensiva di un'applicazione presuppone di norma che esista un perimetro, con l'input dell'utente all'esterno e i componenti interni all'interno, e concentra i controlli sull'attraversamento di quel confine. Un servizio di terze parti si trova, in questo schema, in una posizione ambigua: è esterno all'applicazione ma è stato scelto deliberatamente, è raggiungibile su un canale

che l'applicazione stessa ha configurato, e appartiene spesso a un'azienda la cui reputazione induce a considerarlo affidabile. La conseguenza è che la risposta del fornitore entra nel sistema attraversando un confine che nessuno ha presidiato, perché nessuno lo ha riconosciuto come confine.

Il primo scenario mostra il meccanismo nella sua forma più pura, quella dell'iniezione differita. Un'API si affida a un servizio di terze parti per arricchire gli indirizzi commerciali forniti dagli utenti: l'indirizzo viene inviato al servizio esterno e i dati restituiti sono poi salvati in una base di dati SQL locale. Gli attaccanti usano il servizio di terze parti per memorizzare un payload di SQL injection associato a un'attività commerciale creata da loro; poi si rivolgono all'API vulnerabile fornendo un input specifico che le fa prelevare la loro attività malevola dal servizio esterno, e il payload finisce per essere eseguito dalla base di dati, esfiltrando dati verso un server controllato dall'attaccante [14].

Conviene isolare cosa rende questo scenario diverso da una normale iniezione. L'attaccante non invia mai il payload all'API bersaglio: lo deposita altrove, in un sistema che l'API bersaglio interroga per ragioni proprie. Se la difesa dell'applicazione consiste nel sanificare l'input degli utenti, quella difesa non viene mai attivata, perché l'input dell'utente in questo caso è un innocuo nome di attività commerciale. Il payload arriva dal canale che l'applicazione considera fidato, e arriva in un momento diverso da quello della richiesta che lo ha provocato.

Il secondo scenario mostra il meccanismo della redirectione seguita ciecamente. Un'API si integra con un fornitore terzo per conservare in modo sicuro informazioni mediche sensibili, inviando i dati su connessione cifrata tramite una richiesta a `POST /user/store_phr_record` il cui corpo contiene una sequenza genomica. Gli attaccanti trovano il modo di compromettere l'API del fornitore, che comincia a rispondere a quelle richieste con un `308 Permanent Redirect` verso `attacker.com`; poiché l'API segue ciecamente le redirectioni del fornitore, ripete la richiesta identica, dati sensibili dell'utente compresi, verso il server dell'attaccante [14]. Il dettaglio decisivo è che qui tutto è stato fatto correttamente sul piano della cifratura e della scelta del fornitore: la fuga avviene perché il client HTTP applica un comportamento predefinito, il seguire le redirectioni, che nessuno ha valutato come decisione di sicurezza.

Il terzo scenario è il più breve e il più istruttivo, perché mostra che la terza parte non deve essere compromessa perché l'attacco funzioni. Un attaccante prepara un repository Git denominato `' ; drop db; -;` quando un'applicazione bersaglio esegue un'integrazione con quel repository, il payload di SQL injection viene usato da un'applicazione che costruisce una query SQL ritenendo che il nome del repository sia un input sicuro [14]. Qui il servizio integrato si comporta in modo del tutto corretto: restituisce fedelmente il nome che l'utente ha scelto per il proprio repository. È l'applicazione consumatrice ad aver confuso l'autenticità del canale con l'affidabilità del contenuto, due proprietà che non hanno alcun rapporto fra loro.

Da questi tre casi si ricava la generalizzazione utile. Un servizio di terze parti può veicolare dati malevoli in tre modi indipendenti: perché è stato compromesso, perché è stato usato come deposito da un attaccante che ne è utente legittimo, o perché trasmette fedelmente contenuti che utenti terzi vi hanno immesso. Solo il primo caso richiede un'intrusione, e i restanti due sono alla portata di chiunque possa registrarsi al servizio.

Impatto essenziale OWASP dichiara che l'impatto varia a seconda di ciò che l'API bersaglio fa con i dati prelevati, e che uno sfruttamento riuscito può portare all'esposizione di informazioni sensibili ad attori non autorizzati, a molti tipi di iniezione, o a denial of service [14]. L'impatto tecnico è classificato come severo, e i tre scenari coprono effettivamente tre punti diversi dello spettro: esfiltrazione tramite iniezione nel primo, fuga diretta di dati sanitari nel secondo, distruzione di dati nel terzo.

Ciò che caratterizza l'impatto di questa categoria è che il difetto non ne determina l'entità. Nelle categorie di autorizzazione la gravità è funzione di ciò che il controllo mancante proteggeva; qui è funzione di dove il dato non validato finisce, e quel dato può attraversare diversi componenti prima di arrivare a destinazione. Un'informazione prelevata da un fornitore e memorizzata può essere riletta mesi dopo da un componente diverso, scritto da persone diverse, che la tratta come dato interno perché proviene dalla base di dati dell'organizzazione. La contaminazione, in altri termini, sopravvive alla richiesta che l'ha introdotta.

Un secondo tratto riguarda la scala del danno potenziale. Poiché il vettore passa per una dipendenza condivisa, la compromissione di un singolo fornitore raggiunge

simultaneamente tutte le applicazioni che lo integrano, e ciascuna di esse subisce l'attacco senza che nulla nel proprio perimetro sia stato violato. È la ragione per cui OWASP colloca la categoria nel discorso sugli attacchi alla catena di fornitura, e per cui la valutazione del rischio di un'integrazione non può limitarsi alla funzionalità che essa fornisce.

Mitigazione Le contromisure che OWASP elenca sono quattro e coprono piani distinti [14].

La prima è organizzativa e precede lo sviluppo: nel valutare i fornitori di servizi, occorre esaminarne la postura di sicurezza. È una raccomandazione che si applica al momento della scelta e che richiede di trattare l'integrazione come una decisione architetturale con implicazioni di sicurezza, non come una dipendenza tecnica da aggiungere.

La seconda riguarda il trasporto: tutte le interazioni con l'API devono avvenire su un canale di comunicazione sicuro. È la misura che risponde alla prima delle cinque condizioni di vulnerabilità e che, come osservato, appartiene propriamente alla configurazione dello stack.

La terza è il nucleo della categoria: i dati ricevuti dalle API integrate vanno sempre validati e adeguatamente sanificati prima di essere usati. La formulazione è deliberatamente identica a quella che si applicherebbe all'input dell'utente, ed è proprio questo il punto: la contromisura consiste nell'abolire la distinzione fra le due sorgenti.

La quarta riguarda le redirezioni: va mantenuta una lista di posizioni note verso cui le API integrate possono redirigere quelle proprie, e le redirezioni non vanno seguite ciecamente. Vale la pena notare la forma della prescrizione, che non è un divieto assoluto ma una lista di destinazioni ammesse, riconoscendo che alcune redirezioni sono legittime nel funzionamento ordinario dei servizi.

A queste quattro misure il documento non ne aggiunge altre, ma le cinque condizioni di vulnerabilità ne implicano una quinta, relativa ai timeout e ai limiti sulle risposte, che l'elenco delle prevenzioni non riprende. È un'asimmetria interna al documento: due delle condizioni enunciate non trovano una contromisura corrispondente, e vanno cercate nella trattazione del consumo di risorse.

Una misura ulteriore, che discende dalla natura del difetto piuttosto che dal testo, riguarda la forma della risposta attesa. Se il contratto del fornitore è noto, la risposta può essere validata rispetto a uno schema che ne vincoli tipi, lunghezze e formati, esattamente come si fa per le richieste in ingresso. Ciò non intercetta un payload che rispetti lo schema, come il nome di repository del terzo scenario, ma riduce la superficie a ciò che è sintatticamente ammissibile e trasforma la fiducia implicita in un vincolo dichiarato.

Rilevabilità: stato dell'arte in letteratura OWASP classifica la rilevabilità come media e non fornisce, a differenza di altre categorie, indicazioni su come procedere; la descrizione della debolezza si limita a osservare che gli attaccanti devono identificare i servizi con cui il bersaglio si integra, cioè le sorgenti dei dati, ed eventualmente comprometterli [14]. L'assenza di una metodologia di verifica è essa stessa un'informazione, e conviene ricostruire il motivo.

Sul piano dell'analisi statica il problema si formula in termini precisi, e la formulazione rivela perché gli strumenti esistenti raramente lo intercettano. L'analisi della propagazione della contaminazione, quella su cui si fonda il rilevamento delle iniezioni e dell'SSRF, richiede di specificare un insieme di sorgenti e un insieme di destinazioni; per convenzione consolidata le sorgenti sono i punti in cui entra l'input dell'utente, ossia parametri della richiesta, corpo, intestazioni e cookie. Per API10 la sorgente è un'altra: è il valore restituito da una chiamata HTTP in uscita. Un motore configurato secondo la convenzione ordinaria non considera contaminato quel valore, e di conseguenza un flusso che va dalla risposta di un fornitore a una query SQL non produce alcuna segnalazione, benché sia strutturalmente identico a un flusso che produrrebbe una segnalazione se partisse da un parametro di richiesta.

Ne discende che il rilevamento di questa categoria non richiede una tecnica nuova ma una diversa configurazione di una tecnica nota: l'estensione dell'insieme delle sorgenti alle risposte delle chiamate in uscita. Il costo di questa estensione è però prevedibile, ed è l'aumento dei falsi positivi, poiché la maggior parte dei dati che un'applicazione preleva da servizi esterni viene usata in modi del tutto innocui. Distinguere i flussi rilevanti richiede di sapere quali destinazioni siano sensibili, il che riporta al problema generale della precisione nell'analisi della contaminazione, e

di sapere se fra sorgente e destinazione sia interposta una validazione efficace, che è il problema già incontrato nella trattazione dell'SSRF.

Restano due difetti fra i cinque enunciati che si prestano a un rilevamento più diretto, perché non richiedono di seguire un flusso ma di osservare una configurazione locale della chiamata. Il primo è la redirectione seguita per impostazione predefinita: la maggior parte dei client HTTP la abilita senza che lo sviluppatore debba dichiararlo, sicché il segnale non è la presenza di un'opzione errata ma l'assenza di un'opzione che disattivi il comportamento. Il secondo è il timeout mancante, che ha la stessa struttura. In entrambi i casi si tratta di verificare che una chiamata in uscita dichiararsi esplicitamente una scelta, e l'assenza della dichiarazione equivale all'accettazione del valore predefinito, che è quello insicuro.

Sul piano dinamico la verifica è ostacolata da un problema di controllo. Provare che un'applicazione accetti dati malevoli da un fornitore richiede di fare in modo che il fornitore li restituisca, il che è possibile solo se il servizio consente all'attaccante di immettervi contenuti, come nel primo e nel terzo scenario, oppure sostituendo il fornitore con un doppio controllato durante il collaudo. La seconda strada è praticabile in un ambiente di prova ma verifica il comportamento dell'applicazione, non la sicurezza dell'integrazione reale; la prima dipende dalle caratteristiche del servizio specifico e non si generalizza.

Va infine registrato che la letteratura sottoposta a revisione paritaria dedicata specificamente a questa categoria è pressoché assente, per quanto è stato possibile verificare. Esiste un corpo consistente di lavori sugli attacchi alla catena di fornitura del software, che però riguardano prevalentemente le dipendenze da pacchetti e la loro compromissione, e un corpo altrettanto consistente sull'analisi della contaminazione, che assume la convenzione sulle sorgenti discussa sopra. Il caso specifico dell'API che si fida della risposta di un'altra API si colloca fra i due e non è stato, finora, oggetto di trattazione autonoma. La sua introduzione nella Top 10 soltanto nell'edizione 2023, motivata dall'osservazione di un fenomeno emergente [41], rende plausibile che il ritardo della letteratura rifletta il ritardo con cui il problema è stato riconosciuto come classe a sé.

CAPITOLO 3

Conclusioni

Le conclusioni qui...

Bibliografia

- [1] OWASP Foundation, "A05:2021 – security misconfiguration," https://owasp.org/Top10/2021/A05_2021-Security_Misconfiguration/, 2021, oWASP Top 10:2021. [Online]. Available: https://owasp.org/Top10/2021/A05_2021-Security_Misconfiguration/ (Citato alle pagine 4 e 5)
- [2] SentinelOne, "Cloud misconfigurations: Risks, examples & prevention," <https://www.sentinelone.com/cybersecurity-101/cloud-security/cloud-misconfigurations/>, 2025. [Online]. Available: <https://www.sentinelone.com/cybersecurity-101/cloud-security/cloud-misconfigurations/> (Citato a pagina 4)
- [3] Cloud Computing News, "10 real-life cloud security failures and what we can learn from them," <https://www.cloudcomputing-news.net/news/10-real-life-cloud-security-failures-and-what-we-can-learn-from-them/>, 2025. [Online]. Available: <https://www.cloudcomputing-news.net/news/10-real-life-cloud-security-failures-and-what-we-can-learn-from-them/> (Citato a pagina 5)
- [4] OWASP Foundation, "OWASP API security top 10 – 2023," <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>, 2023. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/> (Citato a pagina 6)

- [5] OWASP API Security Project, “API2:2023 Broken Authentication,” <https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/> (Citato alle pagine 14, 15, 19 e 25)
- [6] OWASP Foundation, “Credential Stuffing,” https://owasp.org/www-community/attacks/Credential_stuffing, oWASP Community Pages. [Online]. Available: https://owasp.org/www-community/attacks/Credential_stuffing (Citato alle pagine 14 e 16)
- [7] Y. Sheffer, D. Hardt, and M. Jones, “JSON Web Token Best Current Practices,” RFC 8725, Internet Engineering Task Force (IETF), BCP 225, feb 2020. (Citato a pagina 15)
- [8] D. Temoshok, D. Proud-Madruga, Y.-Y. Choong, R. Galluzzo, S. Gupta, C. LaSalle, N. Lefkovitz, and A. Regenscheid, “Digital identity guidelines: Authentication and authenticator management,” National Institute of Standards and Technology, NIST Special Publication 800-63B-4, jul 2025. (Citato alle pagine 15 e 25)
- [9] OWASP Foundation, “Authentication Cheat Sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html, oWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (Citato a pagina 15)
- [10] J. Yang, E. Wang, J. Chen, Q. Wang, Y. Zhang, H. Duan, W. Xie, and B. Wang, “Token Time Bomb: Evaluating JWT Implementations for Vulnerability Discovery,” in *Proceedings of the 32nd Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2025. (Citato a pagina 15)
- [11] B. Xu, S. Jia, J. Lin, F. Zheng, Y. Ma, L. Liu, X. Gu, and L. Song, “JWTKey: Automatic Cryptographic Vulnerability Detection in JWT Applications,” in *Computer Security – ESORICS 2023*, ser. Lecture Notes in Computer Science, vol. 14346. Cham: Springer, 2023, pp. 263–282. (Citato a pagina 15)

-
- [12] OWASP API Security Project, “API4:2023 Unrestricted Resource Consumption,” <https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/> (Citato alle pagine 19, 20, 22, 23, 24, 25, 26 e 40)
- [13] —, “API6:2023 Unrestricted Access to Sensitive Business Flows,” <https://owasp.org/API-Security/editions/2023/en/0xa6-unrestricted-access-to-sensitive-business-flows/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa6-unrestricted-access-to-sensitive-business-flows/> (Citato alle pagine 20, 33 e 34)
- [14] —, “API10:2023 Unsafe Consumption of APIs,” <https://owasp.org/API-Security/editions/2023/en/0xaa-unsafe-consumption-of-apis/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xaa-unsafe-consumption-of-apis/> (Citato alle pagine 20, 35, 47, 48, 50, 51, 52 e 53)
- [15] —, “API9:2023 Improper Inventory Management,” <https://owasp.org/API-Security/editions/2023/en/0xa9-improper-inventory-management/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa9-improper-inventory-management/> (Citato alle pagine 20, 28, 31, 32 e 49)
- [16] A. Cha, E. Wittern, G. Baudart, J. C. Davis, L. Mandel, and J. A. Laredo, “A principled approach to GraphQL query cost analysis,” in *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2020)*. ACM, 2020, pp. 257–268. (Citato alle pagine 22, 25 e 27)
- [17] S. A. Crosby and D. S. Wallach, “Denial of service via algorithmic complexity attacks,” in *Proceedings of the 12th USENIX Security Symposium*. Washington, DC: USENIX Association, 2003, pp. 29–44. (Citato a pagina 22)

-
- [18] C.-A. Staicu and M. Pradel, “Freezing the web: A study of ReDoS vulnerabilities in JavaScript-based web servers,” in *Proceedings of the 27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, 2018, pp. 361–376. (Citato alle pagine 23, 25 e 27)
- [19] MITRE Corporation, “CWE-770: Allocation of resources without limits or throttling,” <https://cwe.mitre.org/data/definitions/770.html>, 2026, cWE versione 4.20. Consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/770.html> (Citato alle pagine 23, 24, 26 e 27)
- [20] M. Nottingham and R. T. Fielding, “Additional HTTP status codes,” Internet Engineering Task Force, RFC 6585, apr 2012. (Citato a pagina 25)
- [21] R. Polli, A. Martinez Ruiz, and D. Miller, “RateLimit header fields for HTTP,” IETF HTTPAPI Working Group, Internet-Draft draft-ietf-httpapi-ratelimit-headers-11, may 2026, work in progress. <https://datatracker.ietf.org/doc/draft-ietf-httpapi-ratelimit-headers/>. (Citato a pagina 25)
- [22] R. Chandramouli, “Security strategies for microservices-based application systems,” National Institute of Standards and Technology, NIST Special Publication 800-204, aug 2019. (Citato a pagina 25)
- [23] J. Antunes, N. F. Neves, and P. Veríssimo, “Detection and prediction of resource-exhaustion vulnerabilities,” in *Proceedings of the 19th IEEE International Symposium on Software Reliability Engineering (ISSRE 2008)*. IEEE, 2008. (Citato a pagina 26)
- [24] OWASP API Security Project, “API5:2023 broken function level authorization,” <https://owasp.org/API-Security/editions/2023/en/0xa5-broken-function-level-authorization/>, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa5-broken-function-level-authorization/> (Citato alle pagine 28, 29, 30, 31 e 32)
- [25] —, “API1:2023 broken object level authorization,” <https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>, 2023,

- oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/> (Citato a pagina 28)
- [26] M. Monshizadeh, P. Naldurg, and V. N. Venkatakrishnan, “MACE: Detecting privilege escalation vulnerabilities in web applications,” in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS 2014)*. ACM, 2014, pp. 690–701. (Citato alle pagine 28 e 32)
- [27] MITRE Corporation, “CWE-285: Improper Authorization,” <https://cwe.mitre.org/data/definitions/285.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/285.html> (Citato a pagina 29)
- [28] OWASP API Security Project, “API8:2023 security misconfiguration,” <https://owasp.org/API-Security/editions/2023/en/0xa8-security-misconfiguration/>, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa8-security-misconfiguration/> (Citato alle pagine 30, 32, 38, 39, 40, 41, 42, 43, 44 e 45)
- [29] F. Sun, L. Xu, and Z. Su, “Static detection of access control vulnerabilities in web applications,” in *Proceedings of the 20th USENIX Security Symposium (USENIX Security 11)*. San Francisco, CA: USENIX Association, 2011. (Citato a pagina 32)
- [30] S. Son, K. S. McKinley, and V. Shmatikov, “RoleCast: Finding missing security checks when you do not know what checks are,” in *Proceedings of the 2011 ACM International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2011)*. ACM, 2011, pp. 1069–1084. (Citato a pagina 32)
- [31] V. Atlidakis, P. Godefroid, and M. Polishchuk, “Checking security properties of cloud service REST APIs,” in *Proceedings of the 13th IEEE International Conference on Software Testing, Validation and Verification (ICST 2020)*. IEEE, 2020, pp. 387–397. (Citato a pagina 32)

- [32] F. Liu, Y. Zhang, E. Li, W. Meng, Y. Shi, Q. Wang, C. Wang, Z. Lin, and M. Yang, "BACScan: Automatic black-box detection of broken-access-control vulnerabilities in web applications," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS 2025)*. ACM, 2025, pp. 1320–1333. (Citato a pagina 32)
- [33] V. Felmetsger, L. Cavedon, C. Kruegel, and G. Vigna, "Toward automated detection of logic vulnerabilities in web applications," in *Proceedings of the 19th USENIX Security Symposium (USENIX Security 10)*. Washington, DC: USENIX Association, 2010, pp. 143–160. (Citato a pagina 34)
- [34] G. Pellegrino and D. Balzarotti, "Toward black-box detection of logic flaws in web applications," in *Proceedings of the 21st Network and Distributed System Security Symposium (NDSS 2014)*, San Diego, CA, 2014. (Citato a pagina 34)
- [35] OWASP API Security Project, "API7:2023 server side request forgery," <https://owasp.org/API-Security/editions/2023/en/0xa7-server-side-request-forgery/>, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa7-server-side-request-forgery/> (Citato alle pagine 35, 36, 37 e 48)
- [36] MITRE Corporation, "CWE-918: Server-side request forgery (SSRF)," <https://cwe.mitre.org/data/definitions/918.html>, 2026, cWE versione 4.20. Consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/918.html> (Citato alle pagine 35, 36 e 37)
- [37] M. Wessels, S. Koch, G. Pellegrino, and M. Johns, "SSRF vs. developers: A study of SSRF-defenses in PHP applications," in *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, 2024, pp. 6777–6794. (Citato alle pagine 36 e 37)
- [38] Y. Ji, T. Dai, Z. Zhou, Y. Tang, and J. He, "Artemis: Toward accurate detection of server-side request forgeries through LLM-assisted inter-procedural path-

- sensitive taint analysis,” *Proceedings of the ACM on Programming Languages*, 2025, preprint disponibile come arXiv:2502.21026. (Citato a pagina 38)
- [39] E. Wang *et al.*, “Where URLs become weapons: Automated discovery of SSRF vulnerabilities in web applications,” in *Proceedings of the 45th IEEE Symposium on Security and Privacy (S&P 2024)*. IEEE, 2024. (Citato a pagina 38)
- [40] MITRE Corporation, “CWE-16: Configuration,” <https://cwe.mitre.org/data/definitions/16.html>, 2026, categoria CWE, versione 4.20; mappatura vietata. Consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/16.html> (Citato a pagina 39)
- [41] OWASP API Security Project, “Release notes – OWASP API security top 10 2023,” <https://owasp.org/API-Security/editions/2023/en/0x04-release-notes/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x04-release-notes/> (Citato alle pagine 47 e 54)
- [42] —, “API10:2023 unsafe consumption of APIs – conflicting vulnerability indicators,” <https://github.com/OWASP/API-Security/issues/80>, 2023, issue n. 80 del repository GitHub OWASP/API-Security, marzo 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://github.com/OWASP/API-Security/issues/80> (Citato a pagina 47)

Ringraziamenti

Ringraziamenti qui...